CRIS – CYBER RISK ANALYSIS AND INTELLIGENCE SYSTEM FOR REAL-TIME PHISHING URL AND EMAIL DETECTION

A PROJECT REPORT

Submitted by

VINOTHAGAN K	**230701901**
PRAVEEN S	**230701245**
JANARTHANAN B	**230701125**
POOVARASAN T	**230701228**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING





RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report "CRIS – CYBER RISK ANALYSIS AND INTELLIGENCE SYSTEM FOR REAL-TIME PHISHING URL AND EMAIL DETECTION" is the bonafide work of "VINOTHAGAN K (230701901), PRAVEEN S (230701245), JANARTHANAN B (230701125) and POOVARASAN T (230701228)" who carried out the project work under my supervision Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE
Dr. J. Manoranjini M.E,Ph.D
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai - 602 105

SIGNATURE
Mr. Salman Latheef T A
SUPERVISOR
Assistant Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai - 602 105

Submitted for the Project Viva-Voce Examination held on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

Phishing attacks deceive users into revealing sensitive information through malicious links and fraudulent messages, and they remain difficult to stop because they exploit human judgement rather than software flaws. This project presents CRIS, the Cyber Risk Analysis and Intelligence System, a locally run platform that analyses phishing URLs and messages, explains every verdict and verifies links at the moment they are clicked. URLs are classified by a Random Forest model over thirteen structural features and by a transparent rule-based scoring function, whose outputs are blended with weights of 0.6 and 0.4 into a risk score that maps to Safe, Suspicious or Dangerous. Messages are classified by a TF-IDF and Logistic Regression pipeline supported by alert-word and urgency checks. The platform is built with Flask and provides a Control Center, a URL Inspector, a Mail Analyzer, Insights, History and downloadable PDF reports. The same detection functions are reused by two interception layers: a Chrome Manifest V3 extension that verifies genuine cross-site link clicks, and a Windows link handler registered under the current user that verifies links opened from other desktop applications. Both layers are designed to fail open, so that an unavailable service never blocks normal browsing. The implementation was verified by 34 automated tests, all of which passed, and by sample inputs that show the verdicts behaving as designed. The models are trained on synthetic and hand-written data, so no accuracy for real-world phishing is claimed; evaluation on real labelled datasets is identified as the main direction for future work.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time.

We express our sincere thanks to **DR. J. MANORANJINI M.E, Ph.D,**, Professor and Head of the Department of Computer Science and Engineering for guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Mr. SALMAN LATHEEF T A**, Assistant Professor, Department of Computer Science and Engineering for valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr. S. MADHUSUDHANAN, Ph.D.,** Associate Professor, Department of Computer Science and Engineering for useful tips during our review to build our project.

VINOTHAGAN K
PRAVEEN S
JANARTHANAN B
POOVARASAN T

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
ABSTRACT		iii
ACKNOWLEDGEMENT		iv
LIST OF TABLES		vii
LIST OF FIGURES		viii
LIST OF ABBREVIATIONS		ix
CHAPTER 1	INTRODUCTION	1
1.1	GENERAL	1
1.2	PROBLEM STATEMENT	1
1.3	OBJECTIVES	2
1.4	EXISTING SYSTEM	3
1.5	PROPOSED SYSTEM	3
CHAPTER 2	LITERATURE SURVEY	5
2.1	LITERATURE REVIEW	5
2.1.1	URL-Based Phishing Detection with Machine Learning	5
2.1.2	Ensemble and Deep-Learning Approaches	5
2.1.3	Real-Time and Browser-Based Detection	5
2.1.4	Behavioural Features Based on Redirection	6
2.1.5	Message and E-mail Detection	6
2.1.6	Surveys of Phishing Website Detection	6
2.1.7	Fail-Safe Design of Protection Mechanisms	6
2.2	COMPARISON AND DISCUSSION	7
2.3	CONCLUSION	8
CHAPTER 3	SYSTEM DESIGN	9
3.1	SYSTEM ARCHITECTURE	9
3.2	SYSTEM REQUIREMENTS	11
3.2.1	Software Requirements	11
3.2.2	Hardware Requirements	12
3.2.3	Functional Requirements	12
3.2.4	Non-Functional Requirements	13
3.3	DATA AND PERSISTENCE DESIGN	13
3.4	SYSTEM WORKFLOW	14
3.4.1	Manual URL and Message Analysis	14
3.4.2	Browser-Level Interception	14

3.4.3 Operating-System-Level Interception 16
3.4.4 Reporting and History 17
3.5 URL AND MESSAGE PROCESSING DESIGN 18
3.6 SECURITY AND FAIL-SAFE DESIGN 19
3.7 ROUTE AND INTERFACE ARCHITECTURE 20
CHAPTER 4 PROJECT DESCRIPTION 22
4.1 METHODOLOGY 22
4.2 SYSTEM MODULES 22
4.2.1 URL Analysis Module 22
4.2.2 Rule-Based Risk Scoring Module 23
4.2.3 Message Analysis Module 24
4.2.4 Web Dashboard Module 24
4.2.5 Reporting Module 24
4.2.6 Interception Service Module 25
4.2.7 Browser Extension Module 25
4.2.8 Windows Launcher Module 25
4.2.9 Testing, Packaging and Continuous Integration 26
4.3 DETAILED INTERCEPTION WORKFLOW 26
CHAPTER 5 IMPLEMENTATION AND RESULT DISCUSSION 28
5.1 IMPLEMENTATION 28
5.2 EXPERIMENTAL SETUP 33
5.3 TEST SCENARIOS 34
5.4 EVALUATION METRICS 34
5.5 RESULTS AND DISCUSSION 35
5.5.1 Software Validation 35
5.5.2 Functional Verification with Sample Inputs 36
5.5.3 Feature Importance of the Random Forest 38
5.5.4 Response Time 39
5.6 LIMITATIONS 39
CHAPTER 6 CONCLUSION AND FUTURE WORK 40
6.1 CONCLUSION 40
6.2 FUTURE WORK 40
APPENDIX 1 - URL RISK SCORING RULES 41
APPENDIX 2 - END-TO-END VERIFICATION CHECKLIST 42
REFERENCES 43

LIST OF TABLES

Table 2.1 Comparison of Related Approaches 7
Table 3.1 System Architecture Components 11
Table 3.2 Software Requirements 11
Table 3.3 Hardware Requirements 12
Table 3.4 Functional Requirements 12
Table 3.5 Non-Functional Requirements 13
Table 3.6 Core Data and Persistence Entities 13
Table 3.7 Processing Stages and Outputs 18
Table 3.8 URL Features Extracted by CRIS 19
Table 3.9 Interception Safeguards 20
Table 3.10 Application Routes 21
Table 4.1 Methodology Pipeline of the Proposed System 22
Table 4.2 Synthetic Training Distributions Used for the URL Model 23
Table 5.1 Project Components and Source Files 28
Table 5.2 Experimental Test Scenarios 34
Table 5.3 Evaluation Measures and Their Status 35
Table 5.4 Backend Software Validation Results 36
Table 5.5 Verdicts for Sample URLs 37
Table 5.6 Labels for Sample Messages 37
Table 5.7 Measured Analysis Time per Request 39
Table A.1 URL Risk Scoring Rules 41
Table B.1 End-to-End Project Verification Checklist 42

LIST OF FIGURES

Figure 3.1 Architecture of the Proposed CRIS Platform 10
Figure 3.2 Detection Pipeline of CRIS 14
Figure 3.3 Browser-Level Click Interception Flow 15
Figure 3.4 Sequence of Events for an Intercepted Link Click 16
Figure 3.5 Operating-System-Level Interception Flow 17
Figure 3.6 Risk Score Bands and Fusion Formula 19
Figure 5.1 Control Center Dashboard 29
Figure 5.2 URL Inspector Page 29
Figure 5.3 URL Result Page for a Dangerous URL 30
Figure 5.4 URL Result Page for a Safe URL 30
Figure 5.5 Mail Analyzer Result Page 31
Figure 5.6 Insights Page 31
Figure 5.7 History Page 32
Figure 5.8 Reports Page 32
Figure 5.9 Interstitial Verdict Page for an Intercepted Link 33
Figure 5.10 CRIS Link Guard Extension Options Page 33
Figure 5.11 Feature Importance in the Random Forest URL Model 38

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
API	Application Programming Interface
CI	Continuous Integration
CRIS	Cyber Risk Analysis and Intelligence System
CSS	Cascading Style Sheets
HKCU	HKEY_CURRENT_USER (Windows registry hive)
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
HWND	Window Handle (Windows)
IP	Internet Protocol
JSON	JavaScript Object Notation
LR	Logistic Regression
LSTM	Long Short-Term Memory
ML	Machine Learning
MV3	Manifest Version 3 (Chrome extensions)
NLP	Natural Language Processing
OS	Operating System
PDF	Portable Document Format
RF	Random Forest
SMS	Short Message Service
SVM	Support Vector Machine
TF-IDF	Term Frequency-Inverse Document Frequency
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1
INTRODUCTION

1.1 GENERAL

Phishing is a social-engineering attack in which an adversary impersonates a trusted party in order to make a victim reveal credentials, financial details or other sensitive information, or carry out some other harmful action. Unlike attacks that exploit a flaw in software, phishing exploits human judgement, which is why hardening servers and patching software alone cannot stop it [2]. A single convincing link or message is often enough to compromise an account, and the attack costs the adversary very little to launch at scale.

Two delivery channels dominate in practice. The first is the malicious hyperlink, placed in web pages, advertisements, social-media posts or messages. The second is the deceptive e-mail or text message, which either asks directly for sensitive data or lures the recipient towards a malicious destination. Research on automated detection has produced many machine-learning classifiers for phishing URLs and websites [1], [4], [6], [7], [10] and a smaller number for e-mail [8], [9], but the two channels are usually treated as separate problems, and most classifiers are evaluated offline on curated datasets rather than inside a workflow that can act before a user reaches a malicious page.

This project presents CRIS, the Cyber Risk Analysis and Intelligence System, a locally run platform that brings URL analysis, message analysis, interpretable risk scoring, report generation and real-time link interception together in one system. URLs are classified by a Random Forest model over thirteen structural features and by a transparent rule-based scoring function whose outputs are blended into a single score. Messages are classified by a TF-IDF and Logistic Regression pipeline supported by heuristic checks. Every verdict is shown together with the reasons behind it. The same detection functions are then reused by two interception layers, a Chrome browser extension and a Windows operating-system link handler, so that a link can be checked at the moment it is clicked, whether it is opened from a web page or from another desktop application.

1.2 PROBLEM STATEMENT

Users meet phishing through hyperlinks and through messages, yet the protection available to them has several practical gaps. Many detection systems address only one channel. Many report a bare label or probability without a human-readable justification, which limits

how far an end user can trust or act on the output. Most are evaluated as offline classifiers and never intervene at the moment of the click. Browser-only protection leaves a user exposed when a link is opened from an e-mail client, a chat application or a document viewer, because the operating system, not the browser, opens that link.

A further design question concerns failure. A protection mechanism that blocks ordinary browsing whenever its own service is unavailable is likely to be switched off by the user, which defeats its purpose. Saltzer and Schroeder identify psychological acceptability and fail-safe defaults as core principles for protection mechanisms [12], and these principles shape how CRIS behaves when it cannot be reached. The central problem addressed in this project is therefore how to analyse URLs and messages in one platform, explain every verdict, and verify links at the point of click across both the browser and the desktop, without ever blocking normal browsing when the verification service is not running.

1.3 OBJECTIVES

The objectives of the project are as follows.

- To design and implement a machine-learning classifier for phishing URLs based on thirteen structural URL features.
- To design and implement a text classifier for phishing and spam messages using TF-IDF features and Logistic Regression.
- To combine the machine-learning output with transparent, rule-based scoring so that every verdict (Safe, Suspicious or Dangerous) is accompanied by human-readable reasons.
- To provide a web dashboard with a URL Inspector, a Mail Analyzer, a Control Center, an Insights view, a History view and a Reports view.
- To generate downloadable PDF reports for URL and message analyses.
- To intercept hyperlink clicks in the Chrome browser through a Manifest V3 extension and verify genuine, cross-site, user-initiated navigations before they proceed.
- To extend equivalent verification to links opened from desktop applications through a Windows protocol handler registered without administrator rights.
- To ensure that both interception layers fail open, so that an unavailable verification service never blocks normal use.

- To validate the implementation with automated tests, and to package it with containerisation and continuous-integration support.

1.4 EXISTING SYSTEM

The existing approaches relevant to this work fall into four groups. The first group consists of classifier-centred research prototypes that report strong results on public or purpose-built datasets, including ensemble and deep-learning models for URLs [1], [10], Random Forest and Support Vector Machine models over behavioural URL features [4], comparisons of several classifiers over natural-language-derived features [6], and hybrid URL-based systems [7]. For e-mail, HELPHED combines two base learners in a hybrid ensemble [9]. These works advance classification accuracy, but they are offline studies and each addresses a single modality.

The second group consists of real-time detectors. Rani et al. combine mutual-information feature selection with a Random Forest ensemble for real-time website detection [3], and browser extensions that layer list-based filtering with machine learning have been proposed for real-time protection inside the browser [11]. Their reach ends at the browser: a link opened from another desktop application never passes through them.

The third group is made up of output formats. Many systems return a class label or probability only. Without reasons, a user cannot judge whether a warning is credible, and a developer cannot easily see why a verdict was produced. The fourth group concerns failure behaviour: the reviewed literature does not discuss what the protection layer should do when it is unavailable, although this decides whether it can be used every day.

1.5 PROPOSED SYSTEM

CRIS is a centralised detection platform built on a Flask web application. A single set of detection functions serves three independent entry surfaces: the manual web dashboard, the Chrome extension called CRIS Link Guard, and the Windows link handler. Whichever surface a URL arrives from, it is analysed by the same Random Forest model and rule engine, so the verdict is identical. Messages are analysed by the TF-IDF and Logistic Regression pipeline through the dashboard.

The proposed system has five distinguishing characteristics. It is **hybrid**, because machine-learning probability and rule-based scoring are blended with weights of 0.6 and 0.4. It is **explainable**, because the rule engine records a reason for every indicator it finds and the

Random Forest exposes per-feature importance. It is **multi-surface**, because the dashboard, the browser and the desktop all use the same detection core. It is **fail-open**, because both interception layers open the original link normally whenever the CRIS service cannot be reached. It is **local**, because analysis runs on the user's own machine and no URL or message is sent to an external service. Figure 3.1 in Chapter 3 shows the complete architecture, and Chapter 4 describes each module.

CHAPTER 2
LITERATURE SURVEY

2.1 LITERATURE REVIEW

This section reviews the studies that informed the design of CRIS. They are grouped by the aspect of the problem they address.

2.1.1 URL-Based Phishing Detection with Machine Learning

Sahingoz et al. proposed a real-time anti-phishing system that compares seven classification algorithms over natural-language-processing-derived URL features, and reported that a Random Forest classifier using only the NLP-based features achieved the highest accuracy, 97.98 percent [6]. Lam proposed a framework that applies Random Forest and Support Vector Machine classifiers to behaviour-based URL features and evaluated it on a large corpus of roughly 470,000 URLs collected from Vietnamese phishing-reporting sources [4]. Karim et al. proposed a hybrid machine-learning phishing detection system operating on URL-derived features [7]. The recurring use of Random Forest as a strong and computationally modest base learner for URL features motivates its use in CRIS.

2.1.2 Ensemble and Deep-Learning Approaches

Banik and Sarma applied bagging and stacking over several Long Short-Term Memory models, with mutual-information feature selection over lexical URL features, and reported that the stacking ensemble outperformed the other machine-learning baselines in their study [1]. Yeasmin et al. proposed EnLeM, in which a decision tree, a random forest and a k-nearest-neighbour learner vote after feature selection; the combination was reported to be more accurate and faster to run than the individual classifiers it was compared with [10]. These models show what heavier approaches can achieve, and they illustrate the training and inference cost that CRIS deliberately avoids in order to keep verification fast enough for a single navigation event.

2.1.3 Real-Time and Browser-Based Detection

Rani et al. proposed a real-time phishing-website detector that combines mutual-information-driven feature selection with a Random Forest ensemble classifier [3]. Wirayudha implemented a browser extension with a multi-layer filtering architecture for real-time phishing website detection, in which list-based filters and a machine-learning model are combined, and

evaluated the memory footprint of the extension [11]. These works demonstrate that real-time protection inside the browser is practical. They do not address links opened outside the browser, which CRIS handles through its Windows layer.

2.1.4 Behavioural Features Based on Redirection

Muhammad et al. examined redirection-chain patterns as a behavioural feature category for phishing detection [5]. This is directly analogous to the redirect-pattern feature in the CRIS feature set, which flags a double slash inside the URL path, and it is related to the way the CRIS browser extension unwraps redirect-proxy links such as search-engine redirect wrappers before analysing the real destination.

2.1.5 Message and E-mail Detection

Bountakas and Xenakis proposed HELPHED, a hybrid ensemble learning phishing e-mail detector that runs two base learners in parallel over textual and structural features and combines them through soft voting; it was evaluated on approximately 32,000 legitimate and 3,500 phishing e-mails [9]. Kumar et al. proposed MLSPD, a machine-learning approach that handles spam and phishing detection together [8]. Among the studies reviewed here, these two are the only ones whose scope reaches message content, which makes them the closest counterparts to the message-analysis side of CRIS.

2.1.6 Surveys of Phishing Website Detection

Zieni, Massari and Calzarossa surveyed the body of research on the detection of phishing websites [2]. The survey provides the wider context against which individual techniques can be placed, and it confirms that URL and website classification is the most heavily studied part of the problem.

2.1.7 Fail-Safe Design of Protection Mechanisms

Saltzer and Schroeder describe fail-safe defaults and psychological acceptability as design principles for protection mechanisms: a mechanism should default to a safe state on failure and should not impose a burden that discourages ordinary use [12]. For an everyday, voluntarily installed consumer protection layer, CRIS interprets these principles as fail-open behaviour. If the verification service is unavailable, the original link opens exactly as it would without CRIS, so the protection can never become the reason that browsing stops working.

2.2 COMPARISON AND DISCUSSION

Table 2.1 compares the reviewed approaches with CRIS on the aspects that matter for this project.

Table 2.1 Comparison of Related Approaches

Ref.	Focus	Technique	Relation to CRIS
[1]	URL classification	LSTM ensemble (bagging and stacking), mutual-information selection	Heavier model class; CRIS keeps a light model for low latency
[3]	Real-time website detection	Mutual-information selection with Random Forest ensemble	Same real-time aim; CRIS adds interception surfaces
[4]	URL classification	Random Forest and SVM on behavioural features, about 470,000 URLs	Supports Random Forest for URL features
[5]	Redirection behaviour	Redirection-chain features	Analogous to the CRIS redirect feature
[6]	URL classification	Seven classifiers over NLP-derived URL features; Random Forest best (97.98%)	Supports Random Forest as the URL model
[7]	URL classification	Hybrid machine learning on URL features	Related URL-only approach
[8]	Spam and phishing	Machine-learning spam and phishing detection	Closest to message analysis
[9]	E-mail detection	Hybrid ensemble, two parallel learners, soft voting	E-mail only; CRIS also covers URLs
[10]	Website detection	Decision tree, random forest, k-NN with hard voting	Ensemble alternative for future work
[11]	Browser protection	Browser extension with layered filtering	Browser only; CRIS adds the OS layer
[2]	Survey	Overview of website detection techniques	Context for the whole field
CRIS	URL and message analysis with interception	Random Forest + rule scoring; TF-IDF + Logistic Regression	Integrated, explainable, fail-open

Three observations follow from the comparison. First, Random Forest appears repeatedly as an effective and inexpensive learner for structural and lexical URL features [3], [4], [6], [10], which supports its selection in CRIS. Second, among the studies reviewed, only HELPHED [9] is dedicated to e-mail, and none combines URL analysis and message analysis in a single operational platform that also justifies each verdict. Third, the browser-based works [3], [11] protect only navigation that happens inside the browser. CRIS therefore differs from the literature mainly in scope and integration, and not in the choice of classification algorithm.

2.3 CONCLUSION

The literature shows that ensemble and feature-engineering methods can classify phishing URLs and websites with high accuracy, and that real-time protection inside the browser is feasible. It also shows three gaps: URL and message analysis are rarely combined, verdicts are rarely explained, and protection outside the browser is not addressed. CRIS responds to these gaps with a hybrid, explainable detection core, interception in both the browser and the operating system, and a fail-open design justified by established security principles.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

CRIS follows a layered, service-oriented design. Three entry surfaces send a candidate URL or message to one Flask backend [16], which calls a shared detection core and returns a verdict together with its reasons. Figure 3.1 shows the complete architecture. The web dashboard is used for manual scans of URLs and messages. The Chrome extension watches hyperlink clicks inside the browser. The Windows handler receives links opened from other desktop applications. The extension and the handler both first check that the Flask service is running and, if it is not, open the link normally.

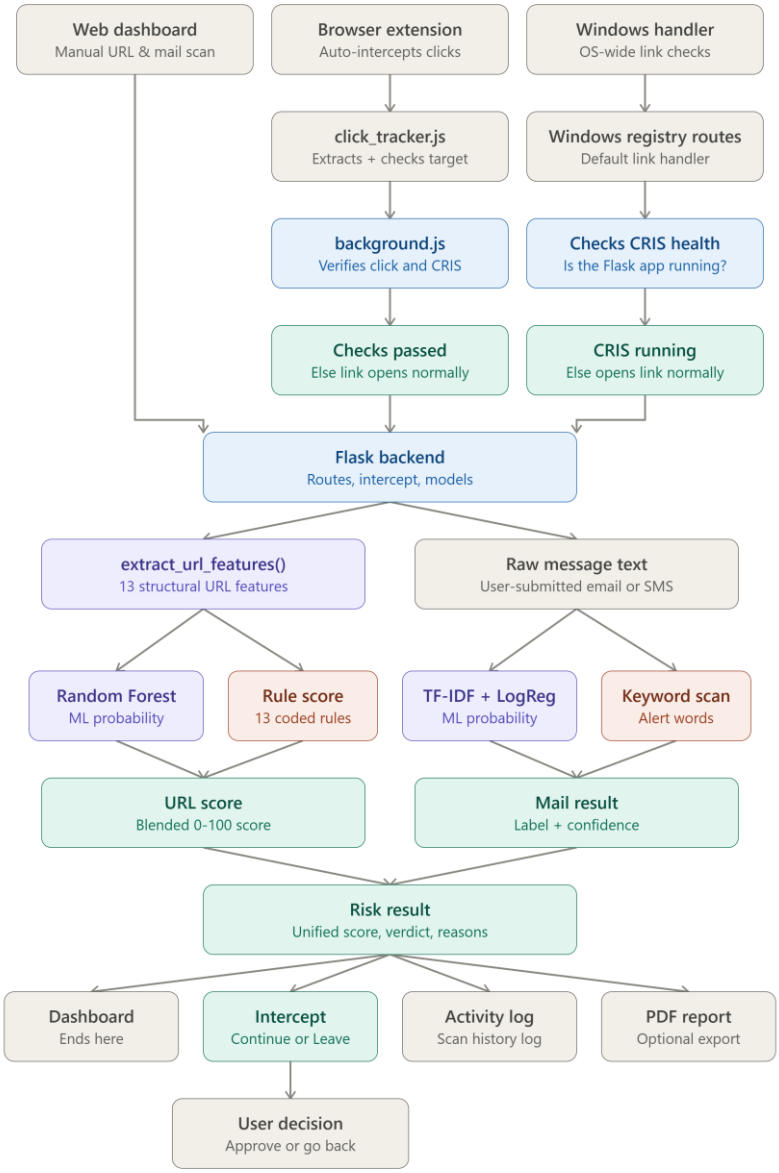


Figure 3.1 Architecture of the Proposed CRIS Platform

Inside the backend, URL analysis and message analysis are independent branches. For a URL, the feature extractor produces thirteen structural features, which feed both the Random Forest classifier and the rule engine. For a message, a TF-IDF vectoriser feeds the Logistic Regression classifier, and a keyword scan supplies the alert words and urgency cues. The branches meet in a unified risk result, which the interface renders, the activity log records, and the report generator can export. Table 3.1 lists the components of the system and the technology used for each.

Table 3.1 System Architecture Components

Layer / Component	Technology / Function
Entry surfaces	Web dashboard (HTML templates), Chrome extension CRIS Link Guard (Manifest V3) and Windows link handler (cris_launcher.py)
Application backend	Flask application package with page routes, JSON API routes and the intercept blueprint
URL analysis	Feature extraction (13 features) and Random Forest classifier from scikit-learn
Rule engine	Deterministic scoring function that records a reason for every indicator found
Message analysis	TF-IDF vectoriser and Logistic Regression classifier, with alert-word and urgency heuristics
Interception service	/intercept verdict page, /intercept/health status endpoint and /intercept/leave focus-restore endpoint
Reporting	ReportLab PDF generation; files stored in app/static/reports
State	In-memory activity log (maximum 50 entries); extension state in chrome.storage and service-worker memory
Quality and delivery	pytest test suite, Dockerfile and Docker Compose, GitHub Actions workflow (Black, isort, mypy)

3.2 SYSTEM REQUIREMENTS

3.2.1 Software Requirements

The software needed to build, run and test CRIS is listed in Table 3.2. The dashboard and the models run on any platform that supports Python, whereas the operating-system-level interception layer requires Windows.

Table 3.2 Software Requirements

Component	Technology / Requirement
Operating system	Windows 10 or 11 (required for the link handler; the dashboard also runs on other platforms)
Programming language	Python 3.10.3
Web framework	Flask
Machine-learning libraries	scikit-learn and NumPy
PDF generation	ReportLab
Browser	Google Chrome with Manifest V3 extension support
Testing	pytest
Code quality	Black, isort and mypy, run by GitHub Actions
Containerisation (optional)	Docker and Docker Compose
Version control	Git and GitHub

3.2.2 Hardware Requirements

CRIS is lightweight because it uses classical machine-learning models and performs all analysis locally. The recommended hardware is given in Table 3.3.

Table 3.3 Hardware Requirements

Component	Recommended Requirement
Processor	64-bit dual-core processor (Intel Core i3 or equivalent) or better
Memory	4 GB minimum; 8 GB recommended
Storage	About 500 MB free for the Python environment and generated reports
Display	1366 x 768 resolution or higher
Network	Not required for analysis; needed only to install dependencies

3.2.3 Functional Requirements

Table 3.4 Functional Requirements

ID	Requirement
FR1	Accept a URL or a message submitted through the web dashboard.
FR2	Classify a URL and return a risk score from 0 to 100, a status, the reasons, a feature breakdown and the machine-learning prediction with its confidence.
FR3	Classify a message and return a label, a confidence value, the spam probability, the flagged words and the reasons.
FR4	Record every analysis in an activity log and show it on the Control Center, History and Insights views.
FR5	Generate PDF reports for URL and message analyses and let the user list and download them.
FR6	Intercept genuine, cross-site, user-initiated link clicks in Chrome and show the CRIS verdict before navigation continues.
FR7	Intercept links opened from other desktop applications through the Windows link handler.
FR8	Show an interstitial page with the verdict, the reasons and the choices Continue to Site and Leave.
FR9	Reject targets that are not http or https URLs, that lack a host name, or that exceed 2048 characters.
FR10	Return focus to the originating desktop application when the user chooses Leave.
FR11	Let the user configure the address of the CRIS service in the extension options.
FR12	Let the user clear the activity history.

3.2.4 Non-Functional Requirements

Table 3.5 Non-Functional Requirements

ID	Requirement	Design Response
NFR1	Availability of normal use	Both interception layers fail open when the service is unreachable.
NFR2	Low latency	Light models and local analysis keep the verdict fast enough for one navigation event.
NFR3	Privacy	All analysis runs locally; no URL or message is sent to an external service.
NFR4	Interpretability	Every verdict lists reasons; the Random Forest exposes feature importance.
NFR5	Stability	A loop guard stops repeated interception of the same destination.
NFR6	Usability	The link handler is registered under the current user, so no administrator rights are needed.
NFR7	Portability	A Dockerfile and Compose file package the dashboard.
NFR8	Maintainability	An automated test suite and a continuous-integration workflow protect the code.

3.3 DATA AND PERSISTENCE DESIGN

CRIS deliberately avoids a database in its current form. The state it keeps is small and short-lived, and the design choice keeps the prototype simple to run. The principal entities and where each is stored are listed in Table 3.6. The most important limitation of this design is that the activity log lives in memory and is lost when the application restarts. This is discussed with the other limitations in Chapter 5.

Table 3.6 Core Data and Persistence Entities

Entity	Content	Storage
Activity entry	id, type (URL or Email), input (first 120 characters), status, score, timestamp	In memory, newest first, at most 50 entries; lost on restart
Analysis result	URL: risk score, status, reasons, feature breakdown, ML prediction, ML confidence, phishing probability. Message: prediction, confidence, spam probability, suspicious words, reasons	Returned with each request
PDF report	Findings and explanations for one analysis	Timestamped file in app/static/reports
Launcher configuration	Path of the real browser (real_browser_path)	cris_config.json beside the launcher
Windows registration	Current-user entries for the http and https protocols and the CRISLauncher application	Windows registry (HKEY_CURRENT_USER)
Extension state	Service address, one-time approvals, health status, click intents	chrome.storage and service-worker memory

3.4 SYSTEM WORKFLOW

3.4.1 Manual URL and Message Analysis

In the manual workflow the user enters a URL in the URL Inspector, or a message in the Mail Analyzer. The Flask route receives the input, calls the matching prediction function, records the outcome in the activity log and renders the result page. The result page shows the verdict, the reasons and, for URLs, the feature breakdown with the importance of each feature. From the result page the user can generate a PDF report. Figure 3.2 shows the detection pipeline that sits behind this workflow.

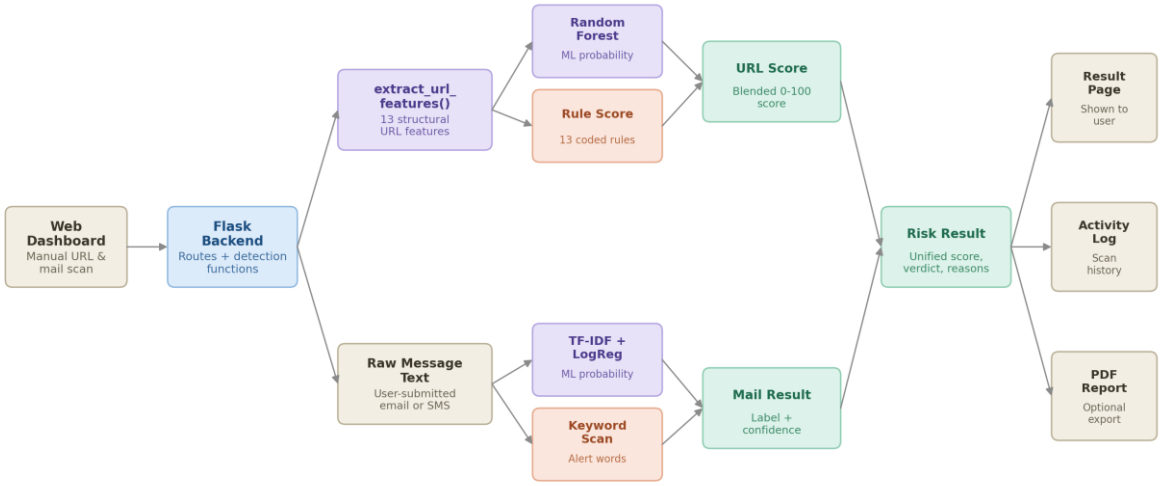


Figure 3.2 Detection Pipeline of CRIS

3.4.2 Browser-Level Interception

The browser extension is a Manifest V3 extension called CRIS Link Guard. A content script runs on every http and https page and listens for click events. It acts only on genuine user clicks, and it resolves the true destination of a link when the link is wrapped by a redirect proxy, for example a search-engine redirect URL. It then tells the extension's service worker which destination the user intended to open.

The service worker listens for top-level navigations that are about to start. It lets a navigation pass untouched unless all of the following hold: the navigation matches a recent genuine user click, the destination is on a different site from the current page, the destination is an ordinary http or https address that is not on the skip list, no valid one-time approval exists for it, the loop guard has not tripped, and the CRIS service answers its health check. When every condition holds, the tab is redirected to the CRIS verdict page. If any condition fails, the

navigation continues normally. Figure 3.3 shows this decision flow, and Figure 3.4 shows the sequence of messages for one intercepted click.

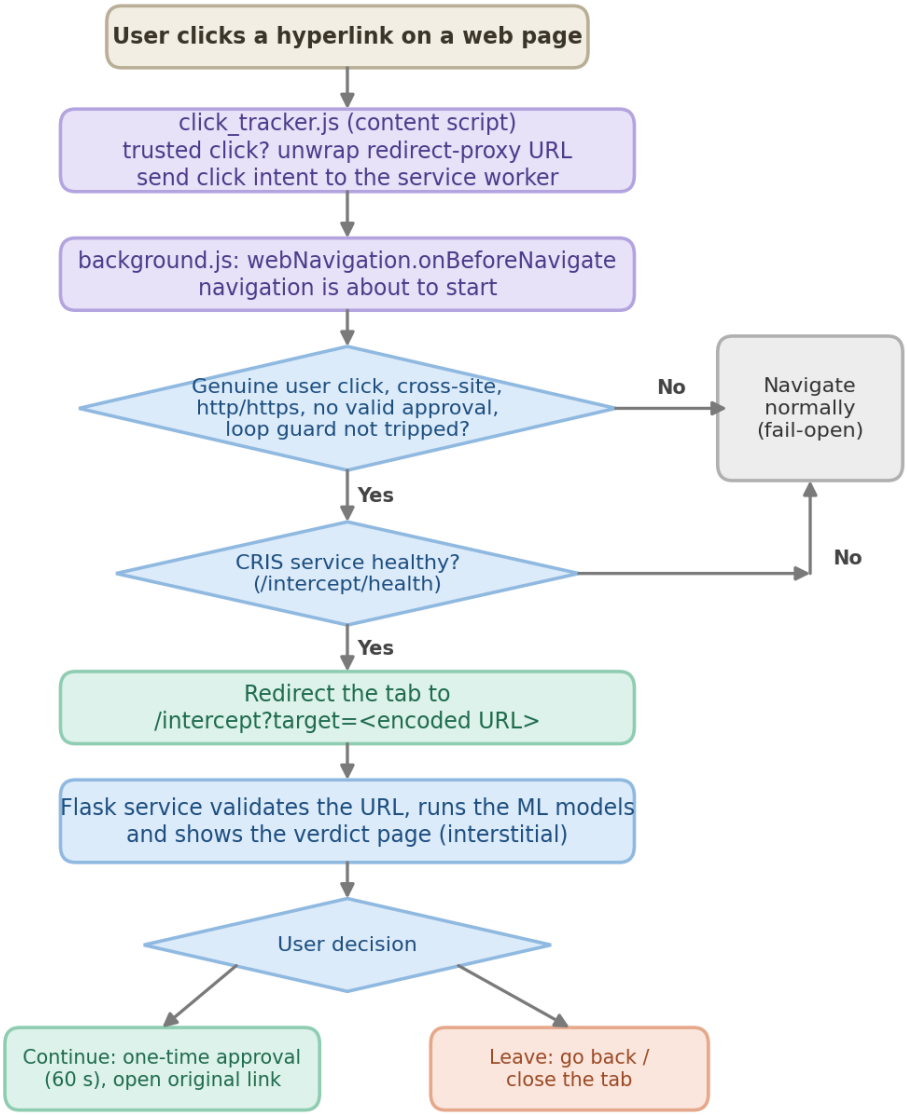


Figure 3.3 Browser-Level Click Interception Flow

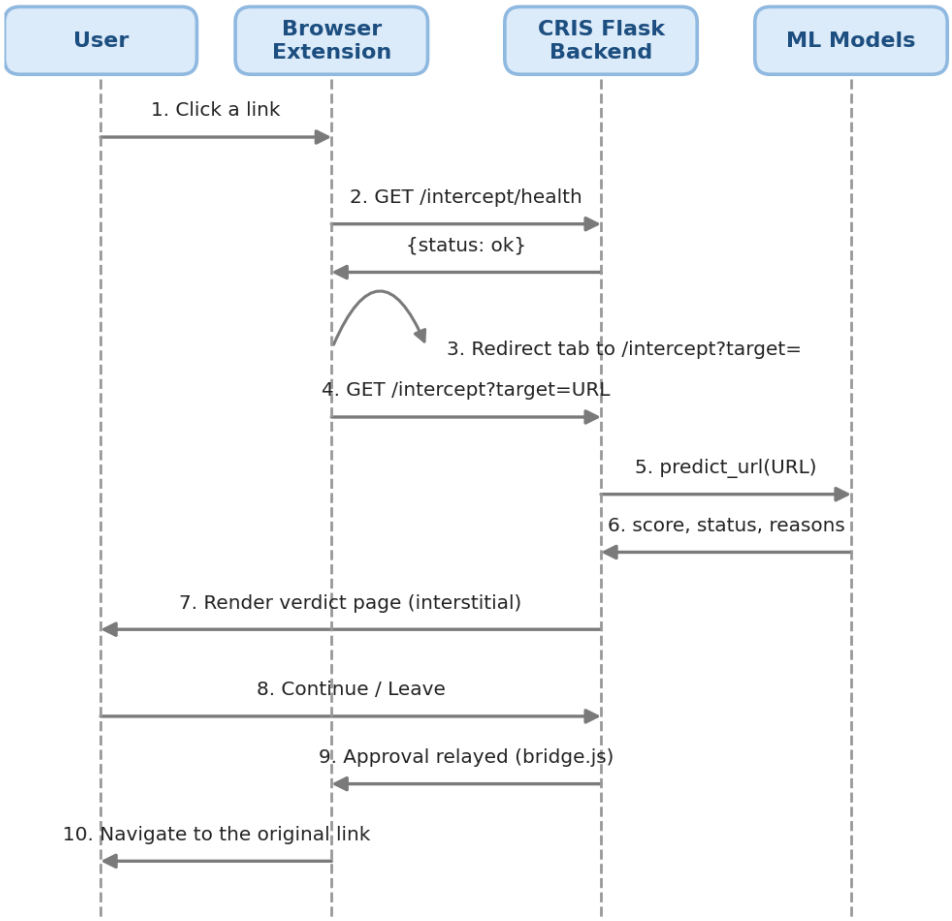


Figure 3.4 Sequence of Events for an Intercepted Link Click

On the verdict page the user chooses Continue to Site or Leave. A small bridge script relays the Continue decision to the extension, which stores a one-time approval for the destination for 60 seconds so that the approved navigation is not intercepted a second time. Leave takes the user back or closes the tab.

3.4.3 Operating-System-Level Interception

A browser extension cannot see links opened from outside the browser. To cover them, CRIS registers a small launcher as the handler for the http and https protocols under the current user's registry hive. When any desktop application opens a link, Windows runs the launcher with the URL. The launcher records the window that was in the foreground, validates the URL, and checks whether the CRIS service is reachable using a health request with a 0.3-second timeout. If the service is running, the launcher opens the real browser at the CRIS verdict page, passing the original URL and the window handle. If it is not running, the launcher opens the original URL in the real browser. Figure 3.5 shows this flow.

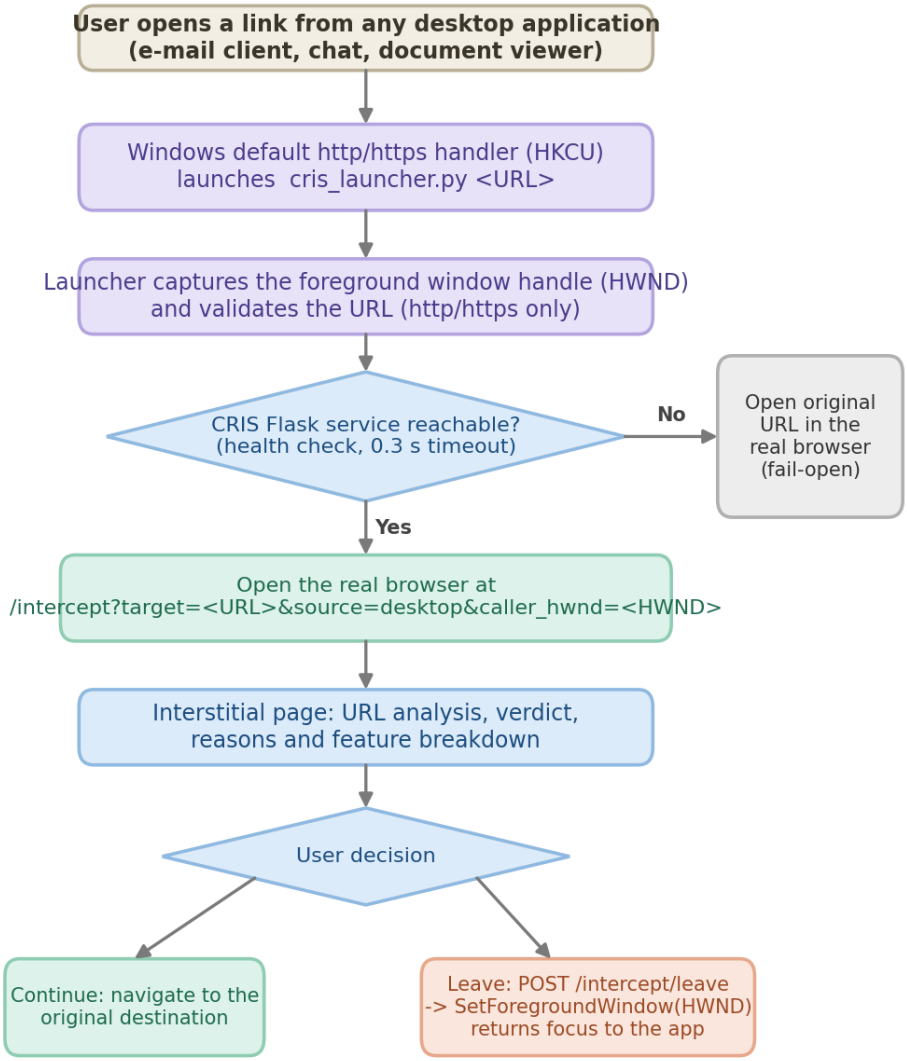


Figure 3.5 Operating-System-Level Interception Flow

Opening the verdict page in a browser window moves keyboard focus away from the application that originated the click. To avoid leaving the user in the browser after a decision to leave, the verdict page posts the stored window handle to the service, which restores and focuses the original window.

3.4.4 Reporting and History

Every analysis, whether it came from the dashboard or from an intercepted link, is added to the in-memory activity log. The Control Center summarises the total number of scans, the number of URL and message scans, the number of threats found and the five most recent entries. The History view lists all retained entries and can be cleared. The Insights view

aggregates keyword frequency and risk distribution over the log. The Reports view lists the generated PDF files and offers them for download.

3.5 URL AND MESSAGE PROCESSING DESIGN

The processing stages for a URL and for a message, and the output of each stage, are summarised in Table 3.7. The thirteen features extracted from every URL are listed in Table 3.8.

Table 3.7 Processing Stages and Outputs

Stage	Description	Output
1. Input validation	Check that the input is present and, for intercepted links, an http or https URL with a host name and at most 2048 characters	Accepted input or an error message
2. Representation	URL: extract 13 structural features. Message: convert to a TF-IDF vector	Feature vector
3. Machine-learning classification	URL: Random Forest. Message: Logistic Regression	Class and probability
4. Rule-based analysis	URL: rule engine adds points for indicators. Message: alert-word, exclamation and urgency checks	Rule score and reasons
5. Fusion	URL: blend the rule score and the ML probability. Message: show both signals	Final score or label
6. Verdict	URL: map the score to Safe, Suspicious or Dangerous. Message: Legitimate or Spam / Phishing	Verdict
7. Presentation and logging	Render the page, record the activity, allow PDF export	Result page, log entry, report

Table 3.8 URL Features Extracted by CRIS

No.	Feature	Description
1	URL Length	Number of characters in the full URL
2	Number of Dots	Occurrences of the dot character
3	Number of Digits	Occurrences of numeric digits
4	Special Characters	Non-alphanumeric symbols other than dot, slash and colon
5	HTTPS Present	1 when the URL uses the https scheme, otherwise 0
6	Contains IP Address	1 when a dotted IPv4 address appears in the URL
7	Suspicious Keywords	Count of 20 watch-list words (for example login, verify, bank, secure, update, password)
8	Subdomains	Number of dot-separated parts of the host name, minus one
9	Has @ Symbol	1 when the character @ appears in the URL
10	Hyphens Count	Occurrences of the hyphen character
11	Path Length	Number of characters in the URL path
12	Has Redirect	1 when a double slash appears inside the path
13	URL Parameters	Number of parameters in the query string

The final URL score combines the two signals as Score = int(0.6 x R + 0.4 x P), where R is the rule-based risk score from 0 to 100 and P is the Random Forest phishing probability expressed as a percentage. The score is capped at 100 and mapped to a verdict: 0 to 30 is Safe, 31 to 60 is Suspicious, and 61 to 100 is Dangerous. Figure 3.6 illustrates the bands. The rules and the points they add are listed in Appendix 1. For messages, the Logistic Regression probability gives the label and confidence, and the alert-word and urgency checks supply the reasons that are shown beside it.

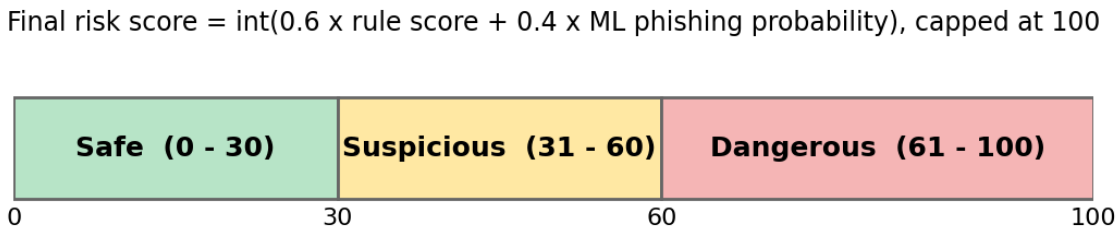


Figure 3.6 Risk Score Bands and Fusion Formula

3.6 SECURITY AND FAIL-SAFE DESIGN

Because CRIS sits between the user and the web, its own behaviour must be safe. Table 3.9 lists the safeguards built into the interception layers and the input checks. The most important design decision is that every failure path ends with the original link opening

normally. A user whose CRIS service has stopped does not notice any change in browsing, which is the property that makes an everyday protection layer acceptable [12].

Table 3.9 Interception Safeguards

Safeguard	Purpose	Behaviour
Genuine click check	Ignore script-triggered navigations	Only trusted user click events are recorded
Cross-site check	Avoid interrupting browsing inside one site	Registrable domains of the page and the destination are compared
Skip list	Leave browser-internal pages alone	chrome:, chrome-extension:, about:, file:, data:, blob:, view-source:, devtools: and Chrome Web Store hosts are never intercepted
Click-intent window	Match a navigation to a click	A click intent is valid for 3 seconds
One-time approval	Do not intercept an approved link twice	An approval lasts 60 seconds
Loop guard	Prevent redirect loops	The same destination in one tab is intercepted at most twice within 15 seconds; a third attempt is let through
Health check	Fail open when CRIS is down	Result cached for 30 seconds in the extension; 0.3-second timeout in the launcher
Scheme allow-list	Reject unsafe targets	Only http and https are accepted
Length limit	Reject abusive input	URLs longer than 2048 characters are refused
Current-user registration	Avoid needing administrator rights	Handler entries are written under HKEY_CURRENT_USER

All analysis runs on the user's own machine, and the extension's host permission is limited to the local CRIS service address. No visited URL or message text leaves the machine as part of the analysis.

3.7 ROUTE AND INTERFACE ARCHITECTURE

The Flask application exposes page routes for the dashboard, JSON routes for programmatic analysis and report generation, and the three routes of the intercept blueprint. They are summarised in Table 3.10.

Table 3.10 Application Routes

Route	Method	Purpose
/	GET	Control Center with scan statistics and recent activity
/url and /url/result	GET	URL Inspector form and URL result page
/mail and /mail/result	GET	Mail Analyzer form and message result page
/insights	GET	Keyword frequency and risk distribution
/history	GET	List of retained scans
/reports	GET	List of generated PDF reports
/settings and /about	GET	Settings and about pages
/predict-url	POST	JSON analysis of a URL
/predict-mail	POST	JSON analysis of a message
/generate-report	POST	Create a PDF report for a URL or message analysis
/download-report/<filename>	GET	Download a generated report
/clear-history	POST	Clear the activity log
/intercept	GET	Verdict page for an intercepted link (browser or desktop)
/intercept/health	GET	Returns status ok; used by the extension and the launcher
/intercept/leave	POST	Restores focus to the originating desktop window

CHAPTER 4
PROJECT DESCRIPTION

4.1 METHODOLOGY

The project followed an incremental approach in which the detection core was built and stabilised first and the interception layers were then added on top of it. The core was kept independent of any particular entry surface, so that the browser extension and the Windows launcher could reuse the same functions without modification. Table 4.1 summarises the stages of the methodology and the outcome of each.

Table 4.1 Methodology Pipeline of the Proposed System

Stage	Activity	Outcome
1. Study and requirements	Review of URL, message and browser-based detection and of fail-safe design; definition of objectives	Requirements in Tables 3.4 and 3.5
2. Feature and rule design	Selection of 13 structural URL features, a 20-word keyword watch-list and the scoring rules	Feature set and rule engine
3. Model construction	Random Forest trained on 1,200 synthetic feature rows; TF-IDF with Logistic Regression trained on 60 sample messages	Two trained models
4. Score fusion	Blend of rule score (weight 0.6) and ML probability (weight 0.4); verdict thresholds	Unified score and verdict
5. Web application	Flask routes, templates, in-memory activity log	Dashboard
6. Reporting	ReportLab PDF generation and download route	PDF reports
7. Interception layers	Intercept blueprint, Chrome extension, Windows launcher and registry handler	Browser and desktop link verification
8. Verification and packaging	pytest suite, Dockerfile, GitHub Actions workflow	34 passing automated tests

4.2 SYSTEM MODULES

4.2.1 URL Analysis Module

The URL analysis module converts a URL into the thirteen features of Table 3.8 and classifies it with a Random Forest. If a URL has no scheme, http is assumed for parsing. The classifier is a scikit-learn Random Forest of 100 decision trees with a maximum depth of 10 and a fixed random seed of 42 [13], [14]. It returns a class (Phishing or Legitimate), a confidence value and the phishing probability, and it exposes the importance of every feature, which the result page displays as the feature breakdown.

The model is trained when the application starts. The training set consists of 1,200 synthetic feature rows, 600 for the safe class and 600 for the phishing class, generated with a fixed seed from the ranges in Table 4.2. The ranges encode the general expectation that legitimate URLs are short, use HTTPS and contain few special characters, whereas phishing URLs tend to be long, contain keywords and digits, use many subdomains and sometimes use an IP address or an at-sign. This synthetic design is a deliberate simplification for a prototype. It is also the most important limitation of the current ML component and is discussed in Chapter 5.

Table 4.2 Synthetic Training Distributions Used for the URL Model

Feature	Safe class (600 rows)	Phishing class (600 rows)
URL Length	10 to 39	50 to 119
Number of Dots	1 to 2	3 to 7
Number of Digits	0 to 2	3 to 14
Special Characters	0 to 1	3 to 9
HTTPS Present	Always 1	1 in 30% of rows
Contains IP Address	Always 0	1 in 40% of rows
Suspicious Keywords	0	1 to 4
Subdomains	1 to 2	3 to 6
Has @ Symbol	Always 0	1 in 30% of rows
Hyphens Count	0 to 1	2 to 5
Path Length	1 to 14	15 to 59
Has Redirect	Always 0	1 in 50% of rows
URL Parameters	0 to 1	2 to 7

4.2.2 Rule-Based Risk Scoring Module

Independently of the classifier, a deterministic function inspects the same features and adds points for each risk indicator, up to a maximum of 100. Examples are a URL longer than 75 characters (20 points), three or more watch-list keywords (30 points), a missing HTTPS scheme (20 points), the presence of an IP address (15 points), an at-sign (10 points) and a double slash in the path (10 points). Every rule that fires appends a plain-language reason, such as 'No HTTPS - insecure connection'. When no rule fires, the reason 'No suspicious indicators detected' is recorded. The complete rule list is given in Appendix 1. The final score blends the rule score with the Random Forest probability as described in Section 3.5.

4.2.3 Message Analysis Module

The message module represents text with a TF-IDF vectoriser [15] configured with unigrams and bigrams, English stop-word removal and a vocabulary limit of 3,000 terms. A Logistic Regression classifier with a regularisation constant of 1.0 and a fixed random seed produces the prediction, the confidence and the spam probability. The classifier is trained at start-up on 60 sample messages, 30 spam or phishing and 30 legitimate, that were written for the prototype.

A set of 28 alert words, including free, winner, urgent, verify, password, suspend, unauthorized and compromised, is searched in the message, and the matches are reported as flagged words. Additional reasons are added when a message contains more than two exclamation marks or urgency phrases such as 'click here', 'act now' and 'limited time'. These heuristics are shown beside the classifier's output so that the user can see why a message was flagged.

4.2.4 Web Dashboard Module

The dashboard is a set of server-rendered pages served by Flask. The Control Center shows the number of total scans, URL scans, message scans and threats found, quick actions to the main tools, and the most recent activity. The URL Inspector and the Mail Analyzer take the input and lead to result pages that show the verdict, the reasons and, for URLs, the feature breakdown. The Insights page summarises keyword frequency and risk distribution, the History page lists earlier scans, and the Reports page lists the generated PDF reports. Settings and About pages complete the interface. A shared base template gives all pages the same navigation and appearance.

4.2.5 Reporting Module

The reporting module uses ReportLab [18] to produce PDF reports. A URL report contains the title Cyber Risk Intelligence Report, the analysis type, a risk assessment table, the findings and explanations, and a feature breakdown table. A message report contains the classification results, the flagged words and the findings. Each report is saved under a timestamped file name in the reports folder and is offered for download through a dedicated route.

4.2.6 Interception Service Module

The interception service is implemented as a Flask blueprint. The verdict route reads the target URL, the source (browser or desktop) and the optional window handle from the query string. It validates the target, rejects it with an error page when it is empty, longer than 2048 characters, not http or https, or without a host name, and otherwise analyses it with the same URL function used by the dashboard. The analysis is added to the activity log and the verdict page is rendered with the Continue to Site and Leave buttons. The leave route restores and focuses the original desktop window when a valid handle is supplied on Windows, and the health route returns a small status message used by both interception layers.

4.2.7 Browser Extension Module

CRIS Link Guard is a Manifest V3 extension [17]. Its manifest requests permission for web navigation, tabs and storage, and grants host access only to the local CRIS service. Two content scripts are declared: the click tracker, which runs on http and https pages, and a bridge script, which runs only on the CRIS verdict page. The click tracker listens for click and auxiliary-click events in the capture phase, ignores events that were not produced by the user, and resolves wrapped redirect links by looking for a destination in query parameters such as q, url, target, dest, destination, link, redirect, to, u and r.

The background service worker holds the decision logic described in Section 3.4.2. It determines whether two addresses are on the same site by comparing registrable domains, with handling for multi-part suffixes such as co.uk and ac.in. It keeps the service address and one-time approvals in extension storage so that they survive a service-worker restart, and it shows the reachability of the service on the extension badge. An options page lets the user change the service address, whose default is http://127.0.0.1:5000.

4.2.8 Windows Launcher Module

Two Python scripts implement the operating-system layer. The registration script finds the user's real browser, saves its path in cris_config.json and writes the registry entries that make the launcher the handler for http and https links. All entries are written under HKEY_CURRENT_USER, so administrator rights are not needed, and an unregister function removes them. The launcher script contains the run-time logic: it reads the foreground window handle with the Windows API, validates the URL, checks the service health, builds the verdict URL with the URL-encoded target, the source and the window handle, and starts the real browser. The real browser is taken from the configuration file, then from the usual installation

paths of Chrome, Edge, Firefox and Brave, and finally from the Windows default shell command.

4.2.9 Testing, Packaging and Continuous Integration

The project includes a pytest suite of 34 tests in three files, which cover the extension decision logic, the intercept routes and the Windows launcher and registration logic. A Dockerfile based on Python 3.10.3 packages the dashboard and exposes port 8888, and a Docker Compose file builds and runs it. A GitHub Actions workflow runs the Black formatter, isort and mypy on every push and pull request.

4.3 DETAILED INTERCEPTION WORKFLOW

This section describes the interception workflows step by step. The browser workflow is:

1. The user clicks a link. The click tracker confirms that the event is a genuine user click, resolves any redirect-proxy wrapper and sends the intended destination to the service worker.
2. When the browser starts a top-level navigation, the service worker ignores skipped schemes and hosts, destinations with a valid one-time approval and destinations that have tripped the loop guard.
3. It checks that a matching click intent was recorded in the last 3 seconds. Navigations without a click, such as typing an address or opening a bookmark, are not intercepted.
4. It compares the registrable domains of the current page and the destination and ignores navigation within the same site.
5. It checks that the CRIS service is reachable, using a result cached for 30 seconds. If the service is down, the navigation continues normally.
6. It redirects the tab to the verdict page with the encoded destination as the target.
7. The Flask route validates the target, runs the URL analysis, logs the activity and renders the verdict, the reasons and the feature breakdown.
8. If the user chooses Continue to Site, the bridge script informs the extension, which stores a 60-second one-time approval and navigates to the original destination. If the user chooses Leave, the user goes back or the tab is closed.

The desktop workflow is:

1. The user opens a link in another application. Windows runs the registered launcher with the URL.
2. The launcher records the foreground window handle and validates that the URL is an http or https address with a host name.
3. The launcher sends a health request with a 0.3-second timeout. If the service does not answer, the launcher opens the original URL in the real browser and finishes.
4. If the service answers, the launcher opens the real browser at the verdict page with the encoded target, the source set to desktop and the window handle.
5. The verdict page is displayed. Continue to Site navigates to the original destination. Leave posts the window handle to the service, which restores focus to the original application.

In both workflows the same URL analysis function produces the verdict, so a given URL receives the same score and reasons whether it was typed into the dashboard, clicked in the browser or opened from another application.

CHAPTER 5
IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION

CRIS is implemented in Python and JavaScript. The detection core, the Flask application and the report generator are Python modules; the browser extension is written in JavaScript; and the Windows layer consists of two Python scripts. Table 5.1 maps each component of the system to its main source files.

Table 5.1 Project Components and Source Files

Component	Main files	Role
Flask application	app/app.py, app/__init__.py	Page routes, JSON routes, activity log
Detection core	app/model.py	Feature extraction, Random Forest, TF-IDF and Logistic Regression, rule engine, score fusion
Interception service	app/intercept.py, templates/intercept.html, static/intercept.js	Verdict page, health and leave routes
Reporting	app/report_generator.py	PDF report generation
Dashboard	12 templates in app/templates, static/style.css, static/script.js	User interface
Browser extension	extension/manifest.json, background.js, click_tracker.js, bridge.js, options.html, options.js	Manifest V3 click interception
Windows layer	cris_launcher.py, register_windows_handler.py	Desktop link interception and registration
Tests	tests/app: test_chrome_extension_logic.py, test_intercept.py, test_windows_interception.py	34 automated tests
Packaging and CI	app/Dockerfile, app/docker-compose.yaml, .github/workflows/main.yaml	Container image and code-quality checks

The user interface of the implemented system is shown in Figures 5.1 to 5.10. Figure 5.1 shows the Control Center, which is the landing page. It displays the number of total scans, URL scans, message scans and threats found, four quick-action cards that lead to the URL Inspector, the Mail Analyzer, the Threat Insights page and the PDF Reports page, and a table of recent activity.

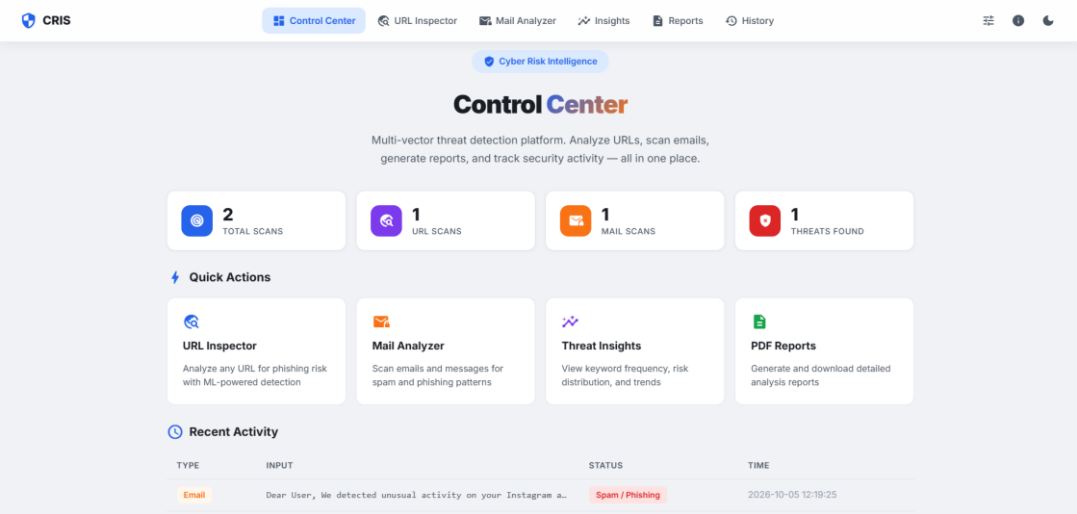


Figure 5.1 Control Center Dashboard

Figure 5.2 shows the URL Inspector, where a URL is entered for analysis. Figures 5.3 and 5.4 show the result pages for a dangerous and a safe URL. Each result page presents the verdict, the risk score, the reasons and the feature breakdown.

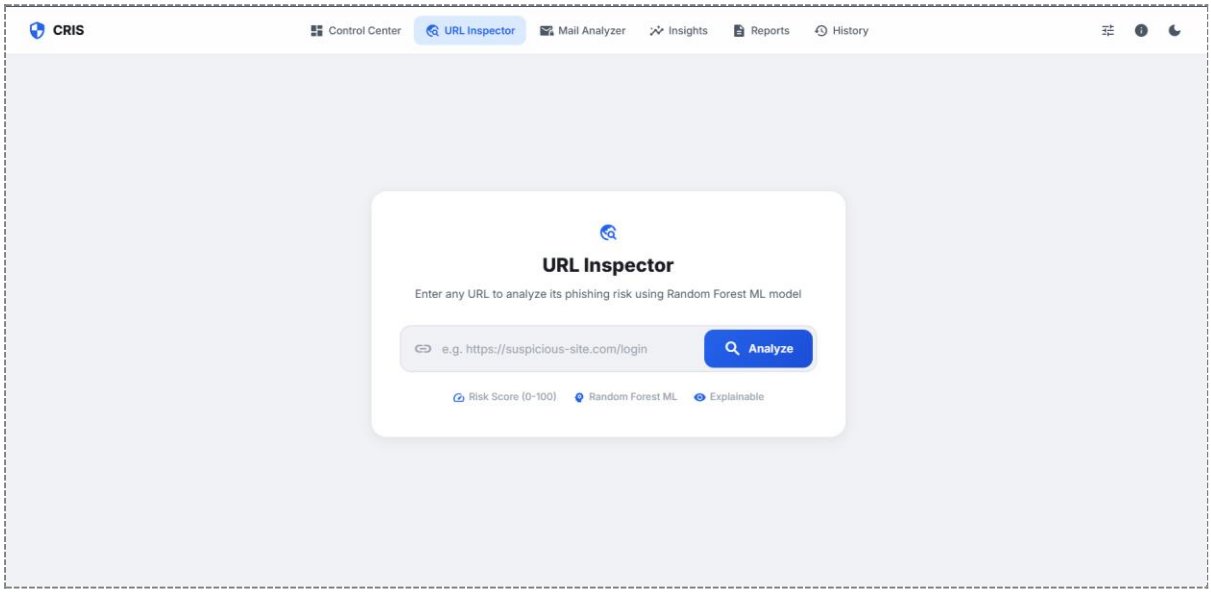


Figure 5.2 URL Inspector Page

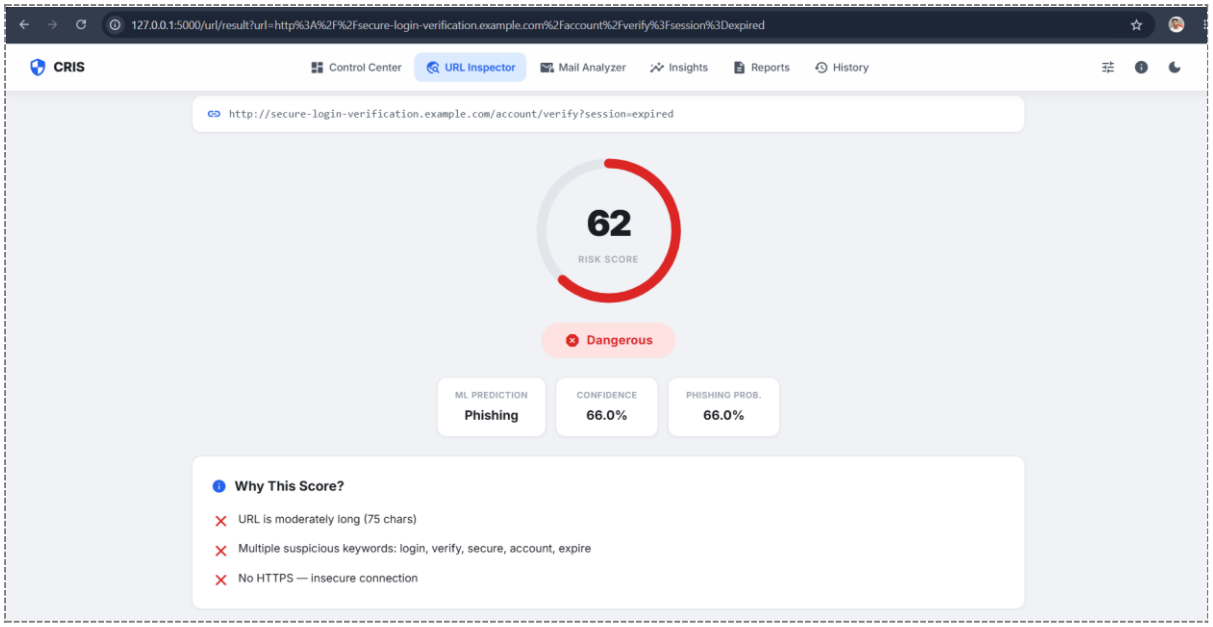


Figure 5.3 URL Result Page for a Dangerous URL

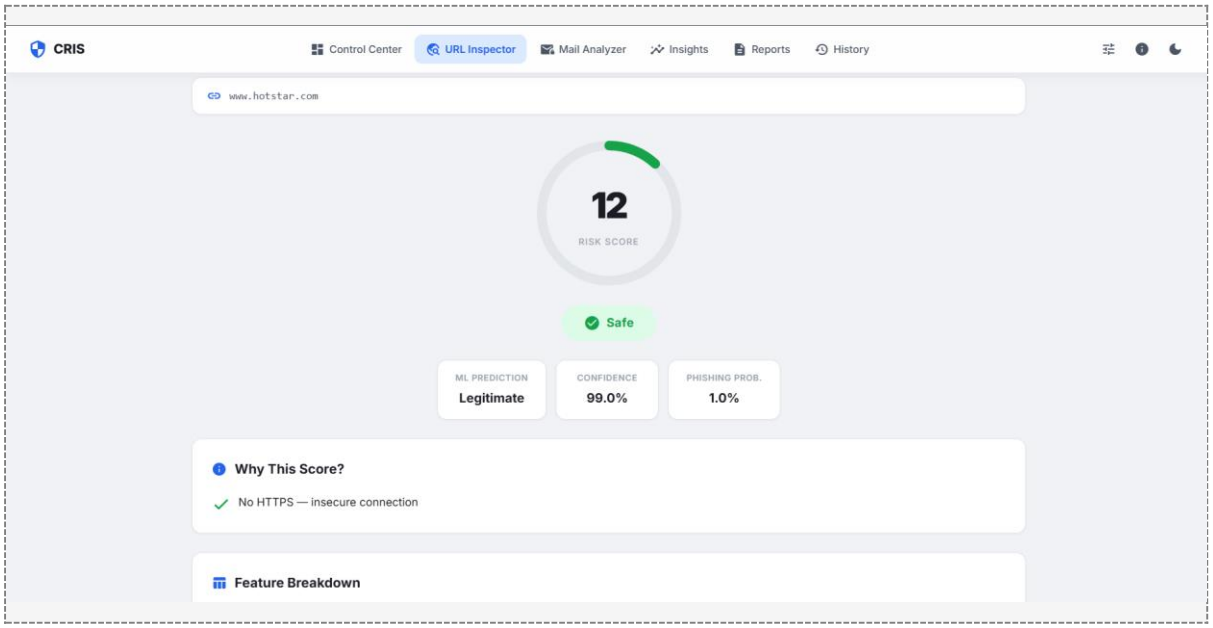


Figure 5.4 URL Result Page for a Safe URL

Figure 5.5 shows the result page of the Mail Analyzer for a phishing-style message, with the label, confidence, spam probability, flagged words and reasons

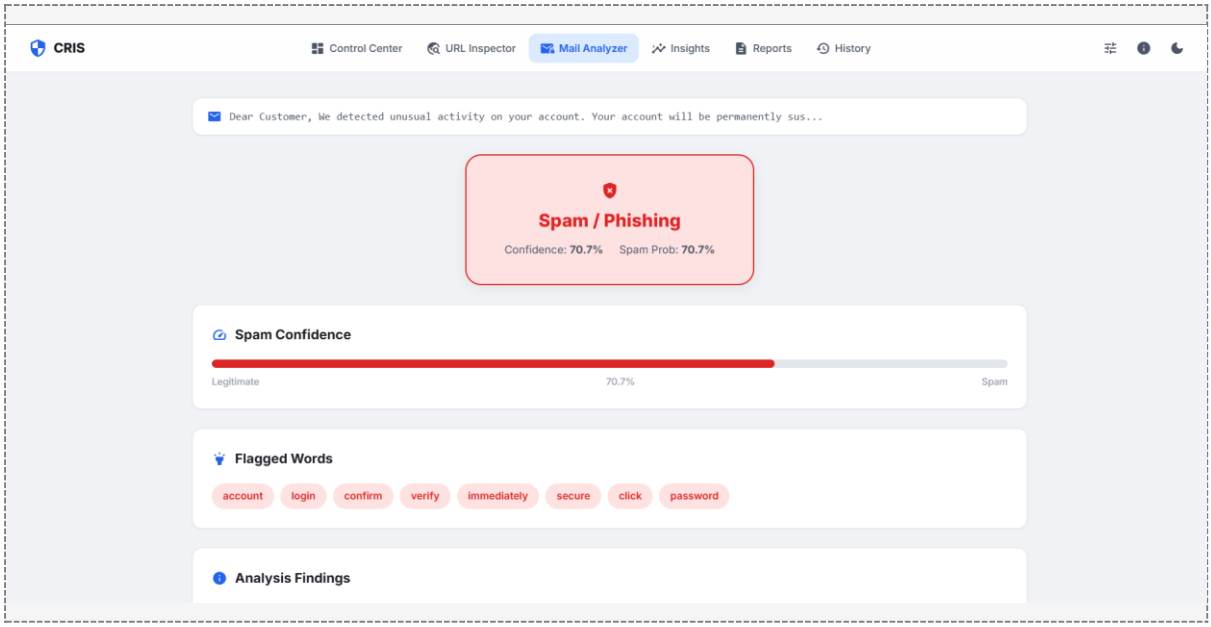


Figure 5.5 Mail Analyzer Result Page

Figures 5.6 to 5.8 show the Insights, History and Reports pages. Insights summarises keyword frequency and risk distribution, History lists the retained scans, and Reports lists the generated PDF files for download.

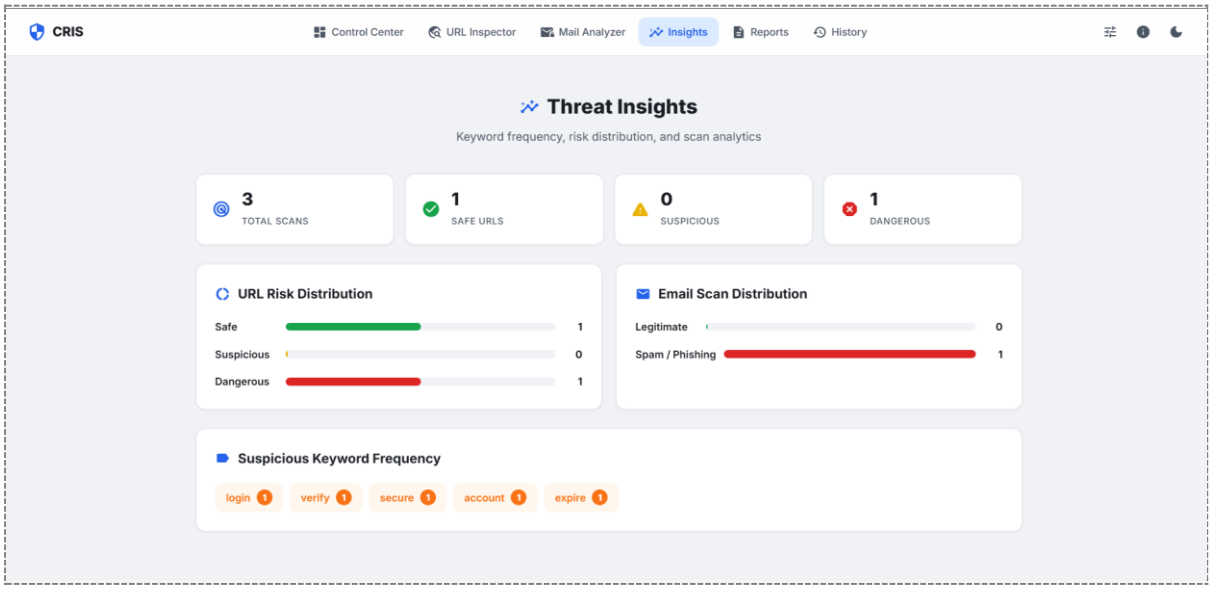


Figure 5.6 Insights Page

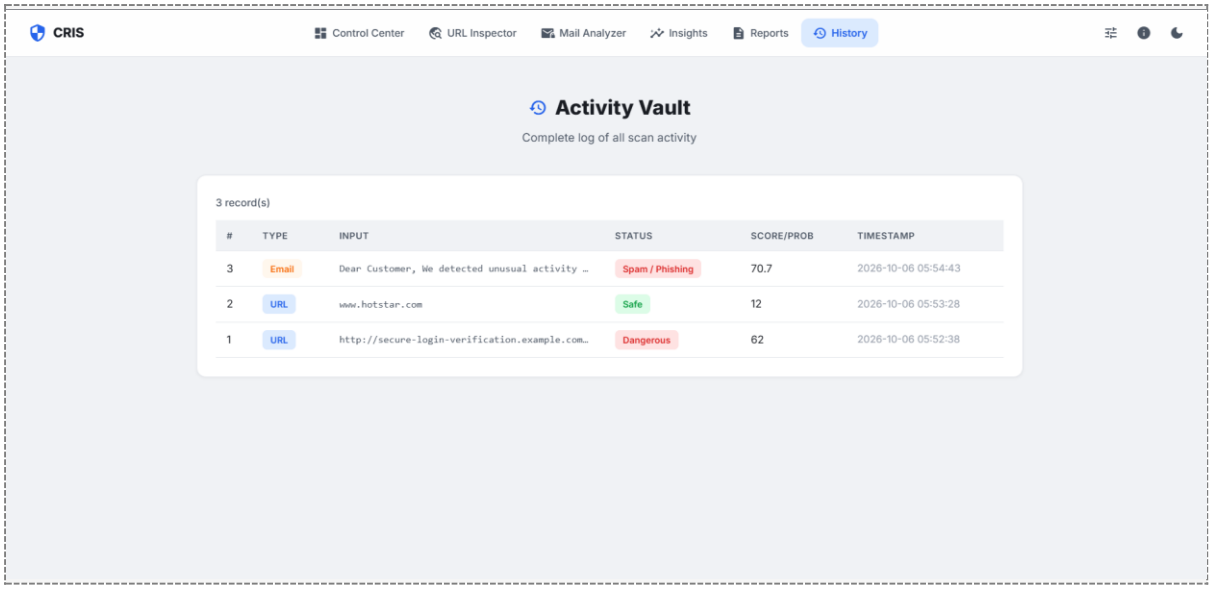


Figure 5.7 History Page

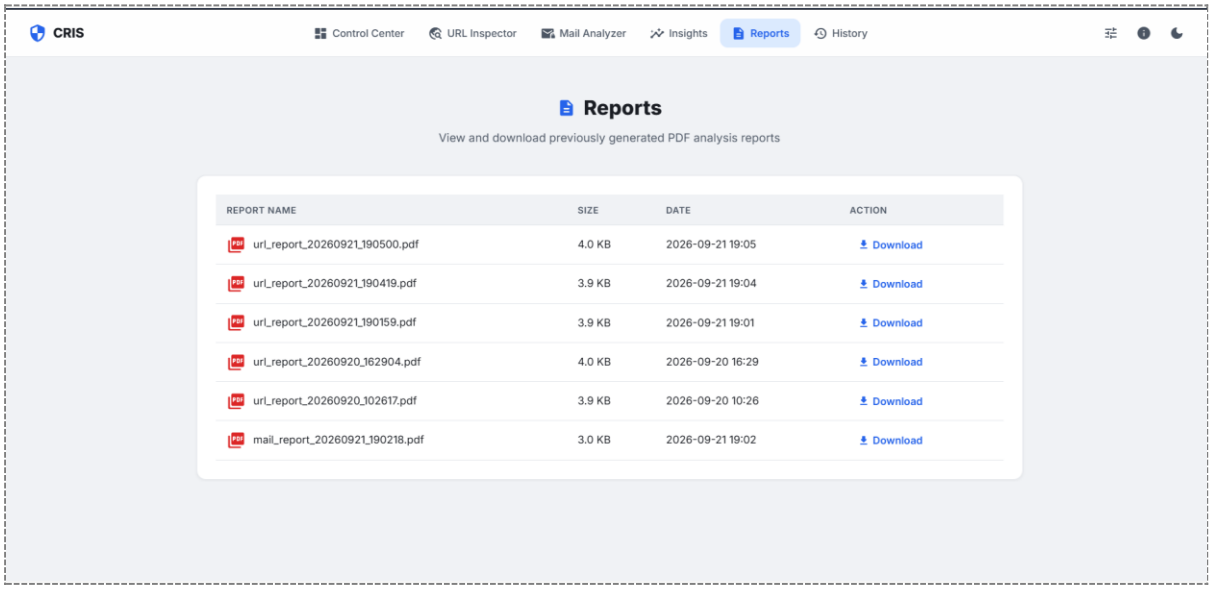


Figure 5.8 Reports Page

Figure 5.9 shows the verdict page that a user sees when a link is intercepted, with the verdict, the reasons and the Continue to Site and Leave buttons. Figure 5.10 shows the options page of the CRIS Link Guard extension, where the service address is configured.

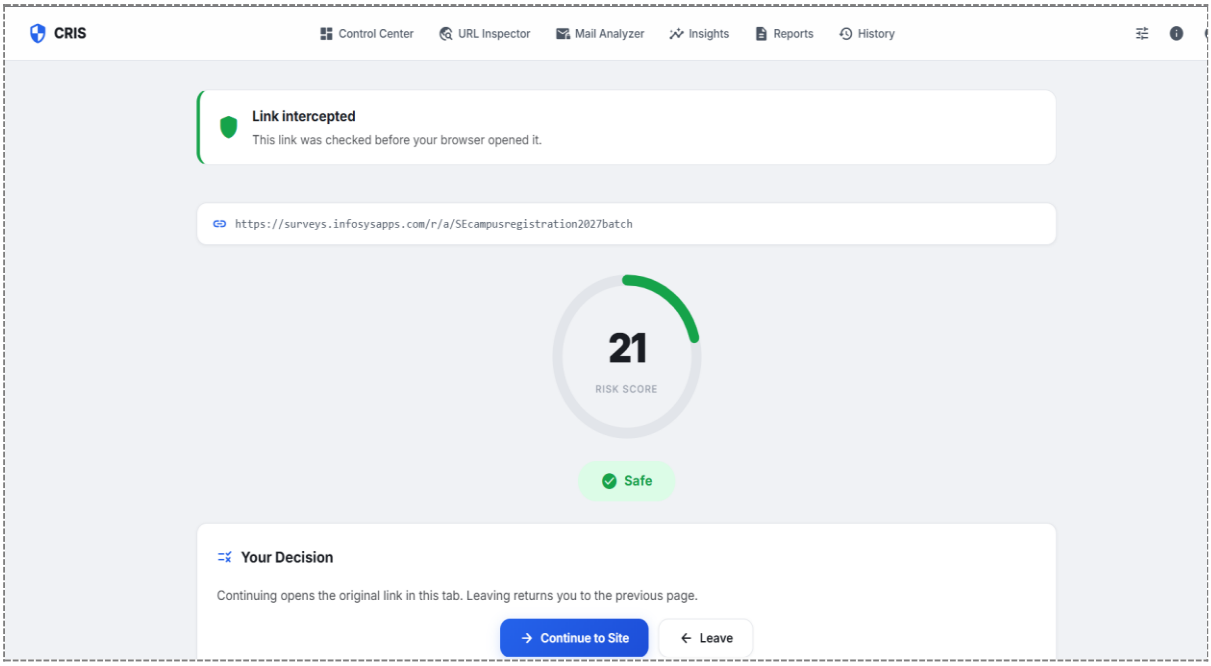


Figure 5.9 Interstitial Verdict Page for an Intercepted Link

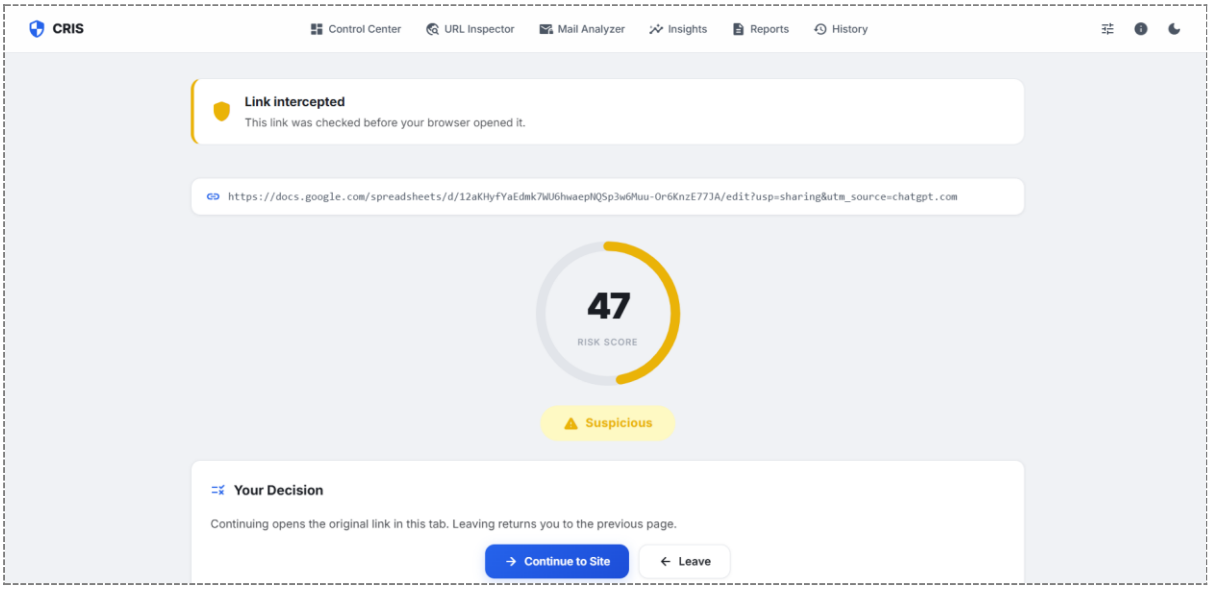


Figure 5.10 CRIS Link Guard Extension Options Page

5.2 EXPERIMENTAL SETUP

The automated tests were executed on a Windows machine using Python 3.10.3 and pytest 9.1.1 inside the project's virtual environment. The test run collected 34 tests. The interception features were exercised against the Flask development server running at 127.0.0.1:5000, which is the default address used by both interception layers.

Both machine-learning models are trained when the application starts, using fixed random seeds, so the sample predictions reported in this chapter can be reproduced with the

same library versions. The sample URLs use the reserved example domains so that no real site is implicated. The sample messages were written for this report, and none of them appears in the 60 messages used to train the message model.

5.3 TEST SCENARIOS

The scenarios in Table 5.2 were chosen to cover the three entry surfaces, the verdict classes and the failure paths. The scenarios that concern the interception logic are covered by automated tests, which are described in Section 5.5.1.

Table 5.2 Experimental Test Scenarios

ID	Scenario	Action or input	Expected outcome
S1	Safe URL	Analyse a well-known HTTPS site	Verdict Safe with a low score
S2	Suspicious URL	Analyse an HTTP login URL	Verdict Suspicious with reasons
S3	Dangerous URL	Analyse a long URL with keywords, subdomains and a redirect	Verdict Dangerous with several reasons
S4	Phishing message	Analyse an urgent account-verification message	Label Spam / Phishing with flagged words
S5	Legitimate message	Analyse an ordinary work message	Label Legitimate
S6	Invalid target	Submit an empty, ftp:, javascript: or over-length target	Request rejected with an error
S7	Internal navigation	Move within the same site, use the address bar or a bookmark	No interception
S8	External link	Click an external link in a mail or chat web page	Redirected to the CRIS verdict page
S9	Redirect wrapper	Click a link wrapped by a search-engine redirect URL	Real destination is analysed
S10	Stale intent	Navigate after the click intent has expired	No interception
S11	Service offline	Open a desktop link while CRIS is not running	Original URL opens normally
S12	Service online	Open a desktop link while CRIS is running	Verdict page opens with source and window handle
S13	Leave decision	Choose Leave on a desktop-sourced verdict page	Focus restored to the original window
S14	Report export	Generate a PDF report from a result	File created and listed under Reports

5.4 EVALUATION METRICS

Two kinds of evaluation are relevant to CRIS, and they must not be confused. Software validation asks whether the application behaves as designed, and it is measured here by the automated test pass rate, by the agreement between verdicts and expectations on sample inputs, and by response time. Classification evaluation asks how well the machine-learning models

separate real phishing from real legitimate content, and it requires a labelled real-world dataset, which this prototype does not yet use. Table 5.3 shows which measures have been obtained and which are planned.

Table 5.3 Evaluation Measures and Their Status

Measure	Meaning	Status
Test pass rate	Share of automated tests that pass	Measured: 34 of 34
Verdict agreement	Whether sample inputs receive the expected verdict	Observed on sample inputs (Tables 5.5 and 5.6)
Response time	Time to analyse one input	Measured (Table 5.7)
Accuracy	Share of all samples classified correctly	Planned; needs a real labelled dataset
Precision and recall	Correctness of alerts and share of phishing caught	Planned; needs a real labelled dataset
F1-score	Balance of precision and recall	Planned; needs a real labelled dataset
False-positive and false-negative rate	Legitimate items wrongly flagged and phishing items missed	Planned; needs a real labelled dataset

5.5 RESULTS AND DISCUSSION

5.5.1 Software Validation

The complete test suite was executed with pytest. All 34 tests passed in 3.23 seconds, which is a pass rate of 100 percent. Table 5.4 groups the tests by file and shows what each group verifies. The extension tests exercise the navigation decision logic in isolation from the browser runtime, the intercept tests exercise the Flask routes through a test client, and the Windows tests exercise the launcher and registration logic with the operating-system calls replaced by test doubles.

Table 5.4 Backend Software Validation Results

Test file	Tests	Passed	Behaviour verified
test_chrome_extension_logic.py	10	10	Internal navigation on ChatGPT and YouTube is not intercepted; external links from WhatsApp and Gmail are intercepted in the same tab and in a new tab; the Google redirect wrapper is unwrapped; address-bar and bookmark navigation is not intercepted; a stale click intent is ignored; redirect chains are handled; tab state is populated at start-up
test_intercept.py	11	11	Health endpoint; empty, javascript:, ftp:, host-less and over-length targets are rejected; a valid target is accepted; the analysis is rendered; a bad scheme is rejected; activity is logged; existing routes keep working
test_windows_interception.py	13	13	Intercept URL building for valid and invalid inputs; real-browser detection; launch through CRIS with a window handle; original URL when CRIS is offline; desktop source and handle on the verdict page; leave endpoint; registry paths and status; saved browser configuration
Total	34	34	Execution time 3.23 s

During verification, one launcher test was found to depend on whether the CRIS service happened to be running when the test was executed. The launcher was behaving correctly, because it is designed to open the original link when the service is unavailable, but the test did not isolate that dependency. The test was corrected by replacing the health check with a test double, and a second test covering the offline case was added, which is why the launcher group now contains both behaviours. This change brought the suite to 34 passing tests.

5.5.2 Functional Verification with Sample Inputs

Table 5.5 shows the verdicts produced for nine sample URLs and Table 5.6 shows the labels for six sample messages. All values were produced by calling the project's own prediction functions. They illustrate how the system behaves and how the rule score and the machine-learning probability combine. They are not a measurement of classification accuracy.

Table 5.5 Verdicts for Sample URLs

No.	Sample URL	Score	Verdict	ML prediction (phishing probability)
1	https://www.google.com/search?q=python	0	Safe	Legitimate (0%)
2	https://github.com/anthropics	0	Safe	Legitimate (0%)
3	https://en.wikipedia.org/wiki/Phishing	0	Safe	Legitimate (0%)
4	http://example.com/login	31	Suspicious	Legitimate (18%)
5	https://accounts.example.com/signin?continue=https://example.com	38	Suspicious	Legitimate (44%)
6	http://paypal.com.account-verify.example.org/update-password/confirm/login.html	78	Dangerous	Phishing (75%)
7	http://free-prize-winner.click-now.example.net/claim?offer=limited&id=99231	76	Dangerous	Phishing (87%)
8	http://192.168.10.45/bank/verify/login.php?user=admin@secure-update.com	93	Dangerous	Phishing (84%)
9	http://secure-login-verify-account.paypal-update.example.com/signin?id=8841&redirect=//evil.com	85	Dangerous	Phishing (87%)

Table 5.6 Labels for Sample Messages

No.	Message (excerpt)	Label	Spam probability
1	Dear User, we detected unusual activity on your Instagram account. Verify your password imme...	Spam / Phishing	68.3%
2	URGENT: Your bank account is locked. Confirm your password and card details immediately to a...	Spam / Phishing	65.9%
3	You have been selected for a limited offer! Claim your free gift card now before it expires!!!	Spam / Phishing	63.9%
4	Hi team, please find attached the minutes of today's project review meeting. Let's meet agai...	Legitimate	35.5%
5	Your order #4412 has been shipped and will arrive on Thursday. Thank you for shopping with us.	Legitimate	48.4%
6	Reminder: the lab record submission deadline is Friday. Please submit the record to the depa...	Legitimate	42.4%

The URL results are consistent with the design. The three well-known HTTPS addresses receive a score of 0 and are Safe. The plain-HTTP login address in sample 4 receives a rule score of 40 (20 points for the missing HTTPS and 20 for the keyword), while the model gives only an 18 percent phishing probability; the blended score is int(0.6 x 40 + 0.4 x 18) = 31, which falls just above the Safe band and is reported as Suspicious. The four remaining URLs combine long hostnames, several keywords, many subdomains, an IP address, an at-sign

or a redirect pattern, and all are rated Dangerous with scores from 76 to 93, supported by phishing probabilities of 75 to 87 percent.

The three phishing-style messages are labelled Spam / Phishing with spam probabilities of 63.9 to 68.3 percent, and the three ordinary messages are labelled Legitimate. The probabilities are moderate rather than extreme, and the second legitimate message reaches 48.4 percent, close to the decision boundary. This is expected from a classifier trained on only 60 short messages, and it shows why a larger real corpus is the first improvement listed in Chapter 6.

5.5.3 Feature Importance of the Random Forest

Figure 5.11 shows the importance that the trained Random Forest assigns to each of the thirteen features. The largest shares belong to the keyword count (17.0 percent), the URL length (16.0 percent), the path length (13.0 percent) and the numbers of dots (11.0 percent), digits and hyphens (10.0 percent each). The binary features for HTTPS (0.5 percent), IP address, at-sign and redirect (0.0 percent each) receive almost no weight.

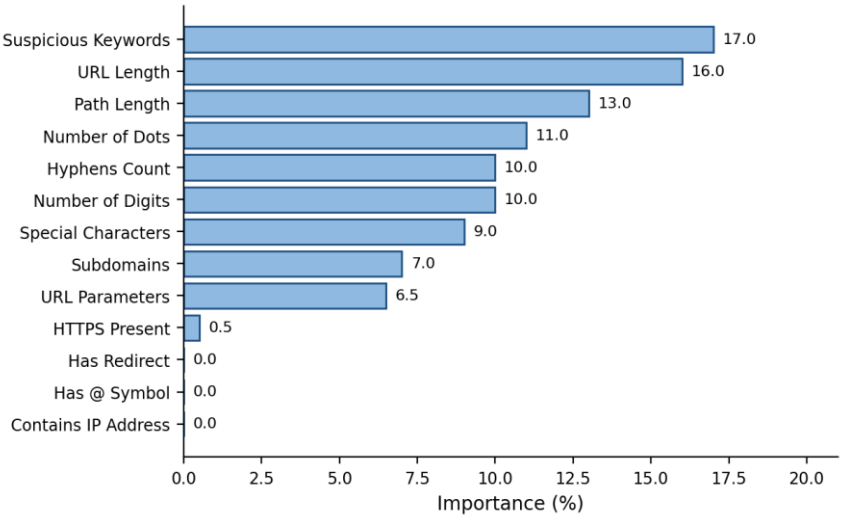


Figure 5.11 Feature Importance in the Random Forest URL Model

These values describe a model trained on the synthetic distributions of Table 4.2, so they show how the forest uses those distributions rather than how phishing behaves in the real world. The binary features receive little weight most likely because the length, count and keyword features already separate the two synthetic classes. This result illustrates the value of the hybrid design: the rule engine adds points whenever an IP address, an at-sign, a redirect pattern or a missing HTTPS scheme appears, so indicators that the learned model underweights still influence the final score and appear in the list of reasons.

5.5.4 Response Time

Response time matters because the interception layers must return a verdict while a navigation is pending. Table 5.7 shows the mean and maximum time of 200 repeated analyses of the same input, measured in the environment used to prepare this report. The timings on the project machine will differ, but the order of magnitude shows that the analysis itself is a small part of the delay a user perceives.

Table 5.7 Measured Analysis Time per Request

Operation	Mean (ms)	Maximum (ms)	Runs
URL analysis (features, Random Forest, rules)	16.38	30.29	200
Message analysis (TF-IDF, Logistic Regression)	0.4	0.51	200

5.6 LIMITATIONS

The results above must be read together with the following limitations of the current prototype.

- **Synthetic and hand-written training data.** The URL model is trained on 1,200 synthetic feature rows and the message model on 60 hand-written messages. No claim is made about accuracy on real-world phishing, and the classification measures of Table 5.3 are still to be obtained on a real labelled dataset.
- **Training at start-up.** Both models are retrained every time the application starts. This is wasteful and means the trained models are not stored as a fixed artefact.
- **In-memory history.** The activity log holds at most 50 entries and is lost when the application restarts. There is no database.
- **Platform scope.** Browser interception supports Chrome, and the operating-system layer supports Windows only.
- **Click-intent heuristic.** The extension recognises a user-initiated navigation by matching a click to a navigation within 3 seconds. Unusual page behaviour could cause a navigation to be missed or, rarely, intercepted unexpectedly.
- **Software validation only.** The automated tests show that the application behaves as designed. They do not measure how well it protects users in the field, which would need a user study.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project designed and implemented CRIS, a platform that analyses phishing URLs and messages, explains every verdict, and verifies links at the moment they are clicked. A Random Forest over thirteen structural features and a transparent rule engine are blended into a single URL score, and a TF-IDF and Logistic Regression pipeline classifies messages. The result is delivered through a web dashboard with a Control Center, a URL Inspector, a Mail Analyzer, Insights, History and PDF reports, and through two interception layers, a Chrome extension and a Windows link handler, that reuse the same detection functions.

The design contributes an integrated treatment of URLs and messages, verdicts accompanied by reasons, interception outside the browser, and a fail-open behaviour that is justified by established security principles. The implementation was verified by 34 automated tests, all of which pass, and by sample inputs that show the verdicts behaving as designed. The report has also been careful about what has not been shown: the models are trained on synthetic and hand-written data, so no accuracy figure for real-world phishing is claimed.

6.2 FUTURE WORK

- Replace the synthetic and hand-written training data with real, labelled URL and message datasets, and report accuracy, precision, recall, F1-score and false-positive and false-negative rates.
- Train the models offline and load them from saved files at start-up, and store the activity history and reports in a persistent database.
- Evaluate whether the ensemble and feature-selection methods surveyed in Chapter 2 improve classification while preserving the low latency and interpretability of the current design.
- Add a fast list-based pre-filter ahead of the classifiers to shorten the time to a verdict for common addresses.
- Extend interception to other browsers and to other operating systems.
- Conduct a usability study of the verdict page and measure the rate of unnecessary interruptions.

APPENDIX 1 - URL RISK SCORING RULES

The rule engine inspects the extracted features and adds the points of Table A.1 for each indicator that applies. The total is capped at 100. The final verdict is then derived from the blended score: 0 to 30 is Safe, 31 to 60 is Suspicious and 61 to 100 is Dangerous.

Table A.1 URL Risk Scoring Rules

Rule	Condition	Points	Reason recorded
Very long URL	Length greater than 75	+20	URL is unusually long
Long URL	Length from 51 to 75	+10	URL is moderately long
Many keywords	Three or more watch-list keywords	+30	Multiple suspicious keywords
Some keywords	One or two watch-list keywords	15 + 5 per keyword	Suspicious keyword(s)
No HTTPS	The URL does not use https	+20	No HTTPS - insecure connection
Excessive dots	More than 4 dots	+10	Possible subdomain abuse
Above-average dots	Exactly 4 dots	+5	Above-average dots
IP address	A dotted IPv4 address is present	+15	URL contains an IP address instead of a domain
Special characters	More than 5	+10	High special character count
At-sign	The character @ is present	+10	Possible obfuscation
Redirect pattern	A double slash in the path	+10	Has redirect pattern
Many digits	More than 6 digits	+5	Many digits in URL
Many parameters	More than 3 query parameters	+5	Many query parameters
No indicator	No rule applies	0	No suspicious indicators detected

APPENDIX 2 - END-TO-END VERIFICATION CHECKLIST

Table B.1 lists the checks used to confirm that the delivered system meets its requirements, and the evidence for each. Checks that concern the interception logic are supported by the automated tests of Section 5.5.1. Checks that concern the user interface are supported by the figures of Section 5.1.

Table B.1 End-to-End Project Verification Checklist

ID	Check	Evidence
V1	Dashboard pages load: Control Center, URL Inspector, Mail Analyzer, Insights, History, Reports	Figures 5.1 to 5.8
V2	URLs receive Safe, Suspicious and Dangerous verdicts with reasons	Table 5.5; Figures 5.3 and 5.4
V3	Messages receive the Legitimate or Spam / Phishing label with flagged words	Table 5.6; Figure 5.5
V4	A PDF report is generated and listed under Reports	Figure 5.8
V5	Invalid targets are rejected by the intercept route and the launcher	pytest: test_intercept.py and test_windows_interception.py
V6	Same-site, address-bar and bookmark navigations are not intercepted	pytest: test_chrome_extension_logic.py
V7	External links opened from Gmail and WhatsApp pages are intercepted	pytest: test_chrome_extension_logic.py
V8	The Google redirect wrapper is unwrapped to the real destination	pytest: test_chrome_extension_logic.py
V9	A stale click intent does not cause interception	pytest: test_chrome_extension_logic.py
V10	The launcher opens the original URL when CRIS is offline (fail-open)	pytest: test_launch_url_when_cris_offline
V11	The launcher routes a desktop link through CRIS with the window handle	pytest: test_launch_url_calls_popen
V12	The leave endpoint returns focus to the originating window	pytest: test_intercept_leave_endpoint
V13	Registry paths and registration status are computed correctly	pytest: test_register_windows_handler_paths_and_status
V14	The verdict page offers Continue to Site and Leave	Figure 5.9; pytest: test_flask_intercept_supports_source_desktop_and_hwnd
V15	The complete automated suite passes	34 passed in 3.23 s

REFERENCES

[1] B. Banik and A. Sarma, "Phishing URL detection using LSTM based ensemble learning approaches," Int. J. Comput. Netw. Commun. (IJCNC), vol. 15, no. 1, pp. 17-33, 2023, doi: 10.5121/ijcnc.2023.15102.

[2] R. Zieni, L. Massari, and M. C. Calzarossa, "Phishing or not phishing? A survey on the detection of phishing websites," IEEE Access, vol. 11, pp. 18499-18519, 2023, doi: 10.1109/ACCESS.2023.3247135.

[3] V. Rani, V. Bardeja, and H. Sharma, "Real-time phishing website detection via mutual information-driven feature selection and random forest ensemble classification," J. Commun., vol. 21, no. 6, pp. 1-8, 2026, doi: 10.43681/JOCS21.06/2605.

[4] N. T. Lam, "Developing a framework for detecting phishing URLs using machine learning," Int. J. Emerg. Technol. Adv. Eng., vol. 11, no. 11, pp. 61-67, 2021, doi: 10.46338/ijetae1121_08.

[5] T. M. Muhammad et al., "Behavioural feature engineering for phishing detection: The case of redirection patterns," J. Behav. Inform., Digit. Humanit. Develop. Res., vol. 11, no. 2, pp. 106-114, 2025, doi: 10.22624/AIMS/BHI/V11N2P210.

[6] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Syst. Appl., vol. 117, pp. 345-357, 2019, doi: 10.1016/j.eswa.2018.09.029.

[7] A. Karim, M. Shahroz, K. Mustofa, S. B. Belhaouari, and S. R. K. Joga, "Phishing detection system through hybrid machine learning based on URL," IEEE Access, vol. 11, pp. 36805-36822, 2023, doi: 10.1109/ACCESS.2023.3252366.

[8] S. Kumar, A. Faizan, A. Viinikainen, and T. Hamalainen, "MLSPD - machine learning based spam and phishing detection," in Computational Data and Social Networks (Lecture Notes Comput. Sci., vol. 11280). Springer, 2018, doi: 10.1007/978-3-030-04648-4_43.

[9] P. Bountakas and C. Xenakis, "HELPHED: Hybrid ensemble learning PHishing email detection," J. Netw. Comput. Appl., vol. 210, Art. no. 103545, 2023, doi: 10.1016/j.jnca.2022.103545.

[10] M. N. Yeasmin, Md. A. R. Refat, B. C. Singh, Z. Alom, Z. Aung, and M. A. Azim, "EnLeM: Ensemble learning-based model to detect phishing websites," EURASIP J. Inf. Secur., vol. 2026, Art. no. 1, 2026, doi: 10.1186/s13635-025-00219-1.

[11] M. R. Wirayudha, "Implementation of a browser extension with multi-layer security filtering architecture for real-time phishing website detection," Eng. Technol. J., vol. 11, no. 5, pp. 10242-10255, 2026.

[12] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proc. IEEE, vol. 63, no. 9, pp. 1278-1308, Sep. 1975, doi: 10.1109/PROC.1975.9939.

[13] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.

[14] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[15] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513-523, 1988, doi: 10.1016/0306-4573(88)90021-0.

[16] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com/

[17] Google, "Chrome extensions documentation (Manifest V3)." [Online]. Available: https://developer.chrome.com/docs/extensions

[18] ReportLab Europe Ltd., "ReportLab open-source PDF toolkit documentation." [Online]. Available: https://docs.reportlab.com/

AI Turnitin

CRIS_formatted.pdf

The Northcap University

Document Details

Submission ID
trn:oid:::1:823695006

Submission Date
Oct 5, 2026, 4:50 PM GMT

Download Date
Oct 5, 2026, 4:52 PM GMT

File Name
CRIS_formatted.pdf

File Size
231.9 KB

7 Pages

4,684 Words

27,128 Characters

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text

Match Groups

76 Not Cited or Quoted 8%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
4% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **76** Not Cited or Quoted 8%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
4% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
ir.vistas.ac.in		2%
2	Internet	
odr.chalmers.se		1%
3	Internet	
naturelightpublications.com		1%
4	Publication	
Modeling Hybrid Feature-Based Phishing Websites Detection Using Machine Lear ...		1%
5	Publication	
A Survey on Machine Learning Techniques for Cyber Security in the Last Decade		1%
6	Internet	
agvolumes.com		1%
7	Publication	
Ambulance route optimization in a mobile ambulance dispatch system using dee ...		1%
8	Publication	
WebGuard: Enhancing Web Security Through an Integrated Developer Platform		<1%
9	Publication	
www.mdpi.com		<1%
10	Publication	
Categorisation of sexual reproductive health short messages texts into thema ...		<1%

11	Publication	ContractFuzzer: fuzzing smart contracts for vulnerability detection <1%
12	Publication	CommentClass: a robust ensemble machine learning model for comment classific ... <1%
13	Internet	journal.ump.edu.my <1%
14	Publication	Fake Job Recruitment Detection Using a Supervised Machine Learning Approach <1%
15	Publication	XAI-PhD: Fortifying Trust of Phishing URL Detection Empowered by Shapley Add ... <1%
16	Internet	eureka.patsnap.com <1%
17	Publication	Explainable Artificial Intelligence (XAI): A Systematic Literature Review on ... <1%
18	Publication	Machine and Deep Learning Models for Stress Detection Using Multimodal Physi ... <1%
19	Publication	Contour-Native Bridge Defect Detection and Compact Digital Archiving with Fr ... <1%
20	Internet	dspace.daffodilvarsity.edu.bd:8080 <1%
21	Publication	A Hybrid Deep Learning Framework for Arecanut Disease Classification Using E ... <1%
22	Publication	A Comprehensive Survey of 5G URLLC and Challenges in <1%
23	Publication	A hybrid super learner ensemble for phishing detection on mobile devices. <1%
24	Internet	mail.jeeemi.org <1%
25	Publication	Improving phishing email detection performance through deep learning with ad ... <1%

26	Internet	
dione.lib.unipi.gr		<1%
27	Publication	
An IoT Toolchain Architecture for Planning, Running and Managing a Complete ...		<1%
28	Publication	
Detection of Phishing Websites and Emails Using Explainable Machine Learning ...		<1%
29	Publication	
A Cognitive BDI–BiLSTM Hybrid Framework for Interpretable Email Threat Detec ...		<1%
30	Publication	
A performance analysis of SHAP and LIME for interpreting boosting models in ...		<1%
31	Internet	
ijmpr.in		<1%
32	Publication	
Advancing Bankruptcy Forecasting With Hybrid Machine Learning Techniques: In ...		<1%
33	Internet	
www.indjcst.com		<1%
34	Publication	
Tight Arms Race: Overview of Current Malware Threats and Trends in Their Det ...		<1%
35	Publication	
Predicting Cognitive Engagement in Online Course Discussion Forums		<1%
36	Publication	
Uncertainties and Design of Active Aerodynamic Attitude Control in Very Low ...		<1%
37	Publication	
Mitigation strategies against the phishing attacks: : A		<1%
38	Publication	
A Lightweight Hybrid MLP-Based Framework for Real-Time		<1%
39	Internet	
linyun.info		<1%
40	Publication	
87535–87562		<1%

41 Publication

IoT-Enabled Smart Waste Management Systems for Smart Cities: A Systematic Re ... <1%

42 Internet

jdmdc.com <1%

43 Publication

A Hybrid Heuristic-Machine Learning Framework for Phishing Detection Using M ... <1%

44 Internet

api.taylorfrancis.com <1%

CRIS: A Hybrid Machine Learning Framework for Phishing URL and Email Detection

Vinothagan K
UG Scholar
230701901@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering.
Rajalakshmi Engineering College
Thandalam, Chennai, India

Janarthanan B
UG Scholar
230701125@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam, Chennai, India

Mr. Salman Latheef T A
Assistant Professor
salmanlatheef.ta@rajalakshmi.edu.in.
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Praveen S
UG Scholar
230701245@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam, Chennai, India

Poovarasan T
UG Scholar
230701228@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam, Chennai, India

Abstract

Phishing remains one of the most enduring and affordable attack vectors in the field of cybersecurity, targeting users with deceptive URLs and fake electronic messages instead of attacking vulnerable software components. A considerable number of prior studies distinguish between phishing URL detection and phishing message detection as two separate challenges and evaluate the algorithms developed separately in carefully crafted single-modality datasets. The paper discusses the detection module of the Cyber Risk Analysis and Intelligence System (CRIS), which represents a hybrid detection architecture based on combining supervised learning and transparent heuristic rule-based approach to analyzing both Uniform Resource Locator (URL) and electronic messages within the same Flask-based web application. Specifically, the CRIS uses a Random Forest classifier for detecting phishing using thirteen structural URL features and risk scoring hand-coded function, while for message detection – a Logistic Regression algorithm with TF-IDF features. The combination of models' results and heuristic reasoning provides a transparent risk conclusion with a corresponding human-readable justification. The paper outlines the detection methodology, system architecture, dashboard implementation, and reporting process and suggests a framework for future dataset-based evaluation of the system. No artificial performance measures are provided; the contribution of the paper is methodological and architectural. The real-time interception of phishing URLs at the moment of user clicking, based on the above detection core, is discussed in a separate work and is out of scope of the paper..

Index Terms

Phishing, Machine Learning, Random Forest, Logistic Regression, TF-IDF, Cyber Security, URLs, E-Mail Security

I. Introduction

Phishing attacks differ from attacks that exploit software vulnerabilities by fooling the victim into believing they are communicating with a trustworthy entity in order to get credentials, financial information, or other data out of the victim, or to get them to perform some other action on behalf of the attacker. Since the attack exploits the human decision-making process, which is not a vulnerability in the technological infrastructure, the strengthening of the latter does little good to thwart it. In any case, only two methods of delivery prove prevalent in practice - links inserted into webpages, advertisements, or other forms of messaging and messages that prompt their recipient for sensitive data or steer them towards a malicious website - but are researched separately from each other as distinct problems, despite sharing a common root.

A considerable number of contributions have made in the form of machine-learning classifiers for phishing URLs and phishing emails, all of which achieve very high results on publicly available datasets [1]-[10]. Nevertheless, there are three practical drawbacks shared by a majority of these works. Firstly, among the ten papers discussed in this essay, eight focus on detecting phishing websites and URLs, one detects emails for phishing attacks [9] and one detects both spam and phishing [8]; however, none, to the best of our knowledge, combines URL and message detection in one system. Second, the great majority of these studies evaluate a classifier offline on a held-out test partition, as for example in [1], [4], [6], [9], [10], rather than as a component embedded in an operational, user-facing workflow. Third, many systems report a classification label without an accompanying, human-readable justification for that label, which limits the extent to which an end user can reasonably trust or act on the output.

This paper presents CRIS (Cyber Risk Analysis and Intelligence System), a detection platform that combines a Random Forest classifier for structural URL analysis and a TF-IDF/Logistic Regression pipeline for message analysis within a single web application, together with a transparent, hand-coded rule-based scoring layer so that every verdict is accompanied by human-readable justification rather than an opaque probability alone. This detection core is also designed to serve as the foundation for a real-time interception workflow, addressed in a separate study outside the scope of the present paper. The present paper is scoped exclusively to the detection core itself, its methodology, implementation, and evaluation plan.

Section II surveys prior ensemble, deep-learning, and real-time approaches to phishing URL and email detection, and

Section III defines the problem and the project's objectives. Section IV presents the proposed system and its architecture, while Section V details the detection methodology for URLs and messages. Implementation, testing, and evaluation are covered in Sections VI through VIII, and the paper closes with discussion, limitations, future work, contributions, and a conclusion (Sections IX-XIII).

II. Related Work

Most prior work bearing on the CRIS detection core applies ensemble or feature-engineering techniques to URL and website classification. A smaller set of studies targets email or messages, and one survey provides wider context; the literature is therefore organized into the four groups below.

A. Ensemble and Feature-Engineering-Based Phishing URL and Website Detection

Several works in this space build ensembles directly on top of lexical or behavioral URL features. In one line of work, bagging and stacking are applied across multiple Long Short-Term Memory (LSTM) models for phishing URL classification, with mutual-information feature selection narrowing the lexical feature set; the stacking variant came out ahead of the other machine-learning baselines tested in that study [1]. Mutual-information-driven feature selection resurfaces in a Random-Forest-based ensemble aimed specifically at real-time phishing-website detection [3] - a design goal that lines up closely with the real-time, point-of-click motivation behind the broader CRIS project. A large-scale study takes a different angle, pairing Random Forest and Support Vector Machine classifiers with behavior-based URL features and testing the approach against roughly 470,000 URLs sourced from Vietnamese phishing-reporting channels [4]. Among seven classification algorithms compared in a real-time anti-phishing system built on natural-language-processing-derived URL features, a Random Forest classifier using only the NLP-based features came out on top, reaching 97.98% accuracy [6]. A hybrid machine-learning system operating purely on URL-derived features is proposed elsewhere [7]. Last but not least, the EnLeM method uses the voting scheme. That means that there are three classifiers: decision tree, random forest, and k-nearest neighbor that make their own vote after the feature selection procedure, and the overall result is supposed to be better and faster than other methods [10]. A pattern worth noting across this group: Random Forest keeps coming out as a strong, computationally cheap base learner for structural or lexical URL features [3], [4], [6], [10], which is a large part of why it anchors the URL-classification component of CRIS (Section V-B).

B. Feature Engineering and Specialized Detection Approaches

Two further studies address more specialized angles. Muhammad et al. examined redirection-chain patterns as a behavioral feature category for phishing detection [5], a feature category directly analogous to the redirect-pattern feature used within the CRIS URL feature set (Table I, feature 12). MLSPD [8] handles spam and phishing detection jointly in one machine-learning framework; within the literature reviewed here, it is the one work besides [9] whose scope reaches message content such as spam, which makes it relevant to CRIS's own combination of URL and message analysis.

C. Phishing Email Detection

HELPHED takes a hybrid ensemble approach to phishing email detection, running two base learners in parallel over a mix of textual and structural features and combining their outputs through soft voting; it was tested on roughly 32,000 legitimate and 3,500 phishing emails [9]. Of the ten papers reviewed for this study, HELPHED is the only one dedicated to email detection; the rest sit on the URL/website side, apart from the joint spam-and-phishing scope of MLSPD [8]. Section II-D returns to this imbalance, and Section XI flags it as a gap worth closing with further literature review.

D. Surveys and Broader Context

Zieni et al. reviewed the existing literature on approaches to detect phishing websites [2]. This review gives wider perspective of the detection approaches, unlike the approaches reviewed in Sections II-A to II-C, which Do not in themselves suggest a detection approach.

E. Positioning of CRIS Relative to Existing Work

The reviewed literature demonstrates that ensemble and feature-engineering-based machine learning, most commonly built around Random Forest, achieves strong classification accuracy for phishing URL and website detection [1], [3], [4], [6], [7], [10], a finding consistent with CRIS's own selection of Random Forest for URL classification. CRIS differs from this literature primarily in scope and integration rather than in classification technique: rather than optimizing URL-classification accuracy in isolation, CRIS combines URL analysis with message and email analysis within a single operational platform, with each pipeline paired with a transparent rule-based scoring layer so that every verdict is accompanied by interpretable justification rather than a bare label. As noted in Section II-C, only HELPHED [9] among the reviewed papers is dedicated to email detection, and no reviewed work combines URL and message analysis within one integrated platform that also justifies each verdict; this combination, together with the rule-based interpretability layer, constitutes the primary point of differentiation for CRIS. CRIS should therefore be understood as complementary to, rather than a replacement for, the classification techniques reported in [1]-[10]; the ensemble and feature-selection methods surveyed in Section II-A in particular represent a direct avenue for improving the URL-classification component of CRIS, as discussed in Section XI.

III. Problem Statement and Objectives

A. Problem Statement

Phishing attacks are delivered to users via two different but loosely connected channels - malicious URLs and fraud electronic messages. Most of the current academic systems used to detect such attacks are developed as offline classifiers and are tested against a single channel. Moreover, the output of these systems is not easily interpretable by a human being. Thus, the problem that we try to solve in this project can be stated as follows:

Is it possible to merge URL- and message-based phishing detection into one system whose decision will be understandable by humans?

B. Objectives

- Create a machine-learning classifier using structural URL properties to detect phishing links.

- Create another machine-learning classifier based on text to identify phishing messages/emails.
- Couple decisions from both classifiers with a clear scoring system based on explicit rules so that all judgments have an easily understandable explanation.
- Cre ate a central interface for the reporting and history functionality.
- Specify a methodical way of evaluation that does not include fake statistics to facilitate further quantitative assessment of the system.
- Ensure that the detection component is independent enough to allow integration into a real-time interception flow at a later stage of development (the process is considered separately)..

IV. Proposed System

CRIS is a centralised detection system implemented using a Flask-based web application that can be accessed via a manual web dashboard. Every URL or message received from the web dashboard undergoes evaluation using the detection algorithms explained in Section V, and the output generated from the evaluation is then recorded and exported.

A. System Architecture

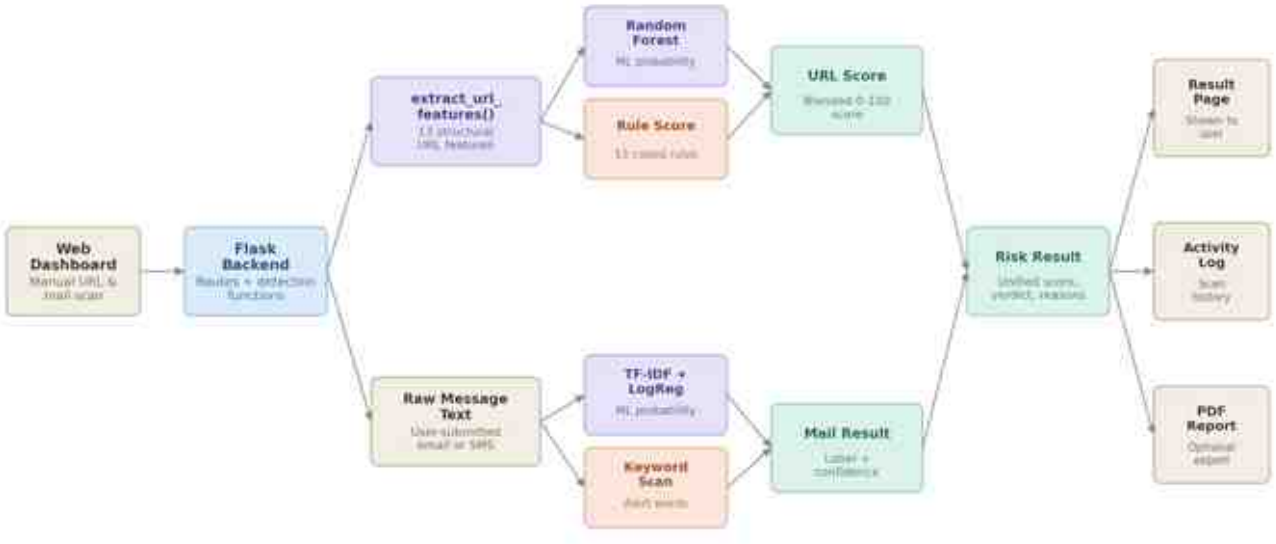


Fig. 1. CRIS detection-core architecture.

B. Manual Analysis Interface

CRIS users will access the tool via a website interface that has two key interfaces; the URL Inspector and the Mail/Message Analyzer. The two tools require a manually inputted URL or message in order to provide its verdict. There are additional screens that make up the interface including the Control Center, which provides quick scan results; Insights, which highlights the frequency of risky keywords; History, which provides previous scan results; and Reports, which provides previous reports.

V. Detection Methodology

URL analysis and message analysis in CRIS are considered independent binary classification tasks that involve both a machine learning model and a rule-based scoring system.

A. URL Feature Engineering

After parsing, all the candidate URLs are represented in a structure feature vector of constant length, shown in Table I below.

TABLE I

URL-Derived Features Supplied to the Classifier

#	Feature	Description
1	URL length	Number of characters in the full URL
2	Dot count	Occurrences of the '.' character
3	Digit count	Occurrences of numeric digits
4	Special chars	Non-alphanumeric symbols present
5	HTTPS	Flag set when the HTTPS scheme is used
6	IP address	Flag set when the host is a raw IP
7	Keyword count	Hits on suspicious terms (e.g., "login")
8	Subdomains	Subdomain levels in the host name
9	'@' symbol	Flag set when '@' appears in the URL
10	Hyphen count	Hyphens appearing in host or path
11	Path length	Characters in the path component
12	Redirect	'//' redirect pattern inside the path

#	Feature	Description
13	Query params	Parameters supplied in the query string

B. URL Classification Algorithm

A feature vector of 13 dimensions is then input to a Random Forest algorithm, which contains 100 decision trees with a maximum depth of 10. Random Forest algorithm was chosen because of the consistency of the performance it has shown as a solid and computationally fast baseline classifier for URL features in previous studies [4], [6], [10].

C. Rule-Based URL Risk Scoring

Independently of the machine-learning classifier, a deterministic rule-based function evaluates the same structural features against a fixed set of heuristics to produce a rule-based risk score in the range 0-100 together with a list of human-readable reasons.

D. Score Fusion and Verdict Classification

The final URL risk score combines the rule-based score with the classifier's predicted phishing probability, expressed as:

$$Score = 0.6 \times R + 0.4 \times (P \times 100) \quad (1)$$

where R is the rule-based risk score and P is the Random Forest's predicted phishing probability. The resulting score is mapped to one of three verdicts - Safe, Suspicious, or Dangerous - using fixed thresholds, and is accompanied by both the triggered rule-based reasons and the per-feature importance derived from the trained Random Forest.

E. Message and Email Feature Representation

The content of messages is encoded through a TF-IDF vectorizer, which uses unigrams and bigrams, removes English stop words, and has a maximum vocabulary size of 3,000, chosen for its lightweight and low-latency nature because of the real-time validation purpose of the whole CRIS project..

F. Message Classification Model

Finally, a Logistic Regression classifier with an inverse regularization value of 1.0 classifies the TF-IDF vector. Like the URL pipeline, the output probability of the classifier is used along with a separate set of heuristic factors such as a list of suspicious keywords, use of too many exclamation marks, and commonly used urgent phrases to give reasons for the classification result..

G. Design Rationale for Model Selection

Selection of Random Forest and Logistic Regression has been made in regard to two project-specific design criteria and not due to any presumption of intrinsic superiority of either method in terms of raw prediction accuracy compared to all other methods discussed in Section II. The first criterion is that both methods are cheap in training and inference in terms of computational costs, which is a requirement because of intended real-time interception workflow, which is a topic of another research, where decision must be made within the budget of one navigation event, while the sequence-based methods, such as the ensemble trained using LSTM in [1], demonstrate the increased training cost and inference overhead introduced by the more complex model. Second, both methods have an easily interpretable internal structure in the form of feature importance in case of Random Forest or coefficient of each term for Logistic Regression with the fixed TF-IDF vocabulary. Such choice allows building rule-based reasoning layer described in Section V-C and V-F without the need for external explainability method. The above selection doesn't prohibit the replacement of the base learner with a stronger model, including the ensemble and feature selection as discussed in Section II-A and II-B [1], [3], [4], [6], [7], [10] on condition that such substitution would preserve the latency and interpretability criteria..

H. End-to-End Processing Pipeline

Regardless of what classifier is used, all submissions go through the same stages of processing: (i) the input data is obtained by the corresponding Flask route; (ii) the input data is converted to its numerical form – a thirteen-dimensional vector for the URL case, or a TF-IDF vector for the message; (iii) the classifier returns the probability of a class; (iv) the independent rule-based module processes the same input and returns the rule score and list of triggering reasons; (v) in the URL case, the probability from the classifier and the rule score are fused using the formula in Section V-D, while in the message case they are just provided side-by-side; (vi) the final decision, confidence scores, and reasons are put into the common result object; and (vii) this result object is simultaneously sent to the requesting web page, added to the activity log, and passed to the PDF report generation module (Section VI-D).

VI. System Implementation

A. Development Approach

First to be developed was the detection engine of CRIS, which defined URL and message classification, scoring based on rules, and the manual dashboard interface, all of which were a stable and independent structure that would later be reused without change by another entry point, the real-time entry point, covered in a different study.

B. Technology Stack

TABLE II
Software Components Used

Component	Technology / Purpose
Server side	Python with the Flask framework
Machine Learning	scikit-learn (Random Forest, Logistic Regression, TF-IDF) and NumPy
Reporting	ReportLab for PDF-based evaluation reports
Frontend	Jinja2 templates with vanilla CSS/JavaScript
Containerization	Docker and Docker Compose
CI / Quality	GitHub Actions running Black, isort, and mypy

C. Application Modules

The features shown in the web dashboard include the following: a Control Center showing the overall level of detection; a URL Inspector and Mail/Message Analyzer to submit manually; an Insights section that provides aggregated information on keywords and risks; a History view that shows past scans; a Reports section to get PDF evaluation reports; and Settings and About pages..

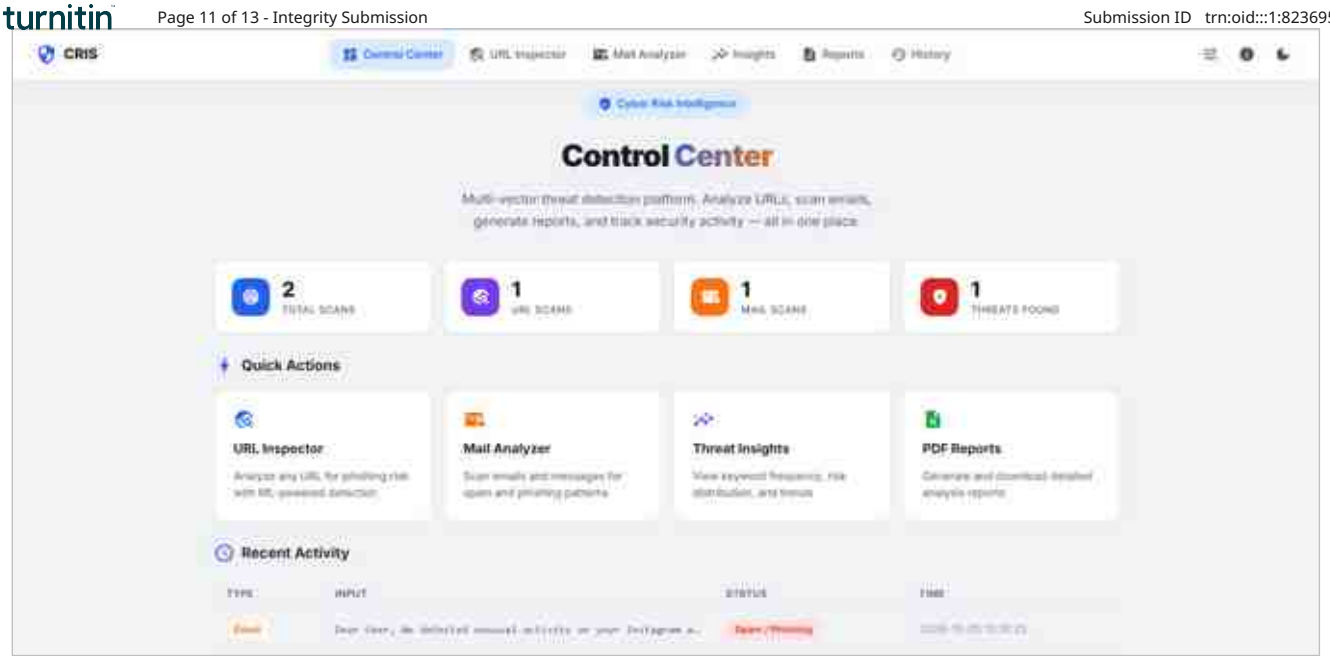


Fig. 2. CRIS Control Center dashboard, showing scan statistics, quick actions, and recent activity.

D. Reporting and Activity Logging

Every evaluation is logged to an activity log that will fill the Control Center view, the History view, and the Insights view and may be optionally exported as a PDF file. The current implementation keeps activities in memory during the lifecycle of the application process but does not store them in a permanent database; this problem is addressed in Section X.

VII. Testing Strategy

Up to now, testing has been done functionally but not formally. All classification and scoring pathways have been put through the system with test URLs and messages, and each run resulted in a full risk analysis - including verdict, confidence scores, and reasons - without any errors.. Unit tests for the individual classifier and feature-extraction functions are not yet in place; they are planned alongside the dataset-driven evaluation described in Section VIII.

VIII. Proposed Evaluation Methodology

No dataset-driven evaluation of the CRIS classifiers has been carried out, which is why this paper quotes no accuracy, precision, recall, or F1 figures. This section instead outlines how such an evaluation should be run.

A. Proposed Dataset Composition

Future evaluation of the URL classifier is intended to use a labeled corpus of real phishing and legitimate URLs, comparable in scale and sourcing to the large, real-world URL collections used in [4] and [6], in place of the synthetic feature distributions currently used for demonstration purposes. Future evaluation of the message classifier is intended to use a real, labeled phishing and legitimate email or message corpus comparable in scale to that used in [9], in place of the small hand-authored example set currently used for demonstration purposes.

B. Proposed Evaluation Protocol and Metrics

The proposed protocol follows conventional supervised-learning evaluation practice: a stratified train/test split (or cross-validation), computation of standard classification metrics on a held-out test partition, and reporting of a confusion matrix for each classifier independently.

TABLE III
Proposed Evaluation Metrics

Metric	Purpose
Accuracy	Fraction of all samples labeled correctly
Precision	Fraction of phishing alerts that were genuine
Recall	Fraction of real phishing cases that were caught
F1-score	Single score combining precision and recall
False-positive rate	Fraction of legitimate items wrongly raised as phishing
False-negative rate	Fraction of phishing cases that slipped through

IX. Discussion

Section II shows that ensembles and engineered-feature classifiers can reach strong accuracy on phishing URL and website data; Sahingoz et al., for instance, report 97.98% accuracy for a Random Forest that uses only natural-language-processing-derived URL features [6]. CRIS does not attempt to exceed such reported figures, and, as stated in Section VIII, does not currently report comparable figures of its own pending a dataset-driven evaluation. The contribution of this work is architectural rather than accuracy-driven: combining URL and message detection within a single platform, with a rule-based interpretability layer that accompanies every machine-learning verdict with human-readable justification - a form of interpretability pursued through a different mechanism than the works reviewed in Section II, the great majority of which report classification performance without an accompanying rule-based justification layer.

based scoring layer in relation to the machine learning classifier. In the URL pipeline, the two signals are numerically combined, meaning that if there were a discrepancy between the heuristic rule scores and those

provided by the classifier, then one gets a numerical score anyway instead of getting a discrepancy signal. In the message pipeline, however, the two signals are shown separately, thus providing insight into any discrepancies between the two scores while sacrificing a unified numerical score. This asymmetric approach to combining the two fusion strategies was deliberately preserved for the purpose of the current paper because unification is an empirical question and depends on the behavior of both on real data, as described in Section VIII.

Another aspect that needs discussion is the dashboard-only interaction model used in this project. While it can be seen as a limitation of the detection core, considering the limited usability of a detection mechanism that can be accessed through submitting a URL or a message, this is a conscious decision for the current stage. The fact is that a separate study should be conducted, in which an automatic entry surface would be developed. The reason is that, due to the separation from the entry surface, the functions of the detection core can be reused without any modifications..

X. Limitations

The current implementation carries several limitations that define the scope of the present contribution and distinguish an architectural demonstration from a production-ready, field-validated system.

First, the URL classifier is currently trained on synthetically generated feature distributions rather than on labeled real-world URLs, and the message classifier is trained on a small, hand-authored set of example messages rather than on a real phishing-message corpus. Consequently, no claim is made regarding real-world generalization until the evaluation described in Section VIII is completed on the datasets proposed there.

Second, both classifiers are retrained in memory each time the application starts, rather than being trained offline once and loaded from a persisted model artifact at startup. This does not affect the correctness of a given run, but it is computationally wasteful at scale and means the exact trained model is not guaranteed to be identical between successive runs, which complicates reproducibility of any future reported result.

Third, activity history and generated reports are retained only in memory for the duration of the running application process, rather than in a persistent database; this history is lost whenever the process is restarted, which is acceptable for the current demonstration scope but would not be acceptable for a deployed system expected to retain an audit trail across sessions.

Finally, no quantitative classification results are yet available, as stated in Section VIII, and this paper is scoped exclusively to the detection core itself; a real-time, interactive extension of this system is left entirely to future, separately reported work.

XI. Future Work

Several directions follow from the limitations identified in Section X. The most immediate is replacing the synthetic and hand-authored training data with real-world labeled URL and message datasets and executing the evaluation methodology proposed in Section VIII, since this is the step that would convert the architectural claims made in this paper into measured results.

A second direction is persistence: training the classifiers offline and loading them as saved model artifacts at application startup, together with introducing a persistent database for activity history and reporting data, in place of the current in-memory implementations.

A third direction is evaluating whether the ensemble and feature-selection techniques surveyed in Section II improve classification performance when substituted into the CRIS pipelines, while preserving the latency and interpretability properties discussed in Section V-G.

Finally, the detection core presented in this paper is intended to serve as the foundation for a separately reported extension of the system toward real-time, point-of-click protection; the design and evaluation of that extension are outside the scope of the present paper.

XII. Contributions

Taken together, this work contributes the following.

- An integrated architecture combining independently trained URL and message phishing classifiers with a shared, transparent rule-based scoring layer.
- A score-fusion formulation combining machine-learning probability with rule-based scoring to produce an interpretable, human-readable verdict.
- A centralized dashboard, activity-logging, and PDF-reporting workflow for reviewing detection activity.
- A structured, non-fabricated evaluation methodology intended to guide future quantitative assessment.
- A self-contained detection core designed to serve as the foundation for a later, separately reported extension of the system.

XIII. Conclusion

This paper has described CRIS, which pairs Random-Forest-based URL classification with TF-IDF/Logistic-Regression-based message classification, and ties both to a transparent rule-based scoring layer inside a single Flask-based platform. In keeping with the academic integrity requirements for this work, no performance results have been fabricated; a structured evaluation methodology is proposed in their place. The detection core described here is meant to underpin a real-time interception extension, which is the subject of a separate, future study.

Acknowledgment

The authors thank Rajalakshmi Engineering College, and in particular its Department of Computer Science and Engineering, for the infrastructure and support provided during this work, and their project guide and faculty mentors for ongoing feedback.

References

[1] B. Banik and A. Sarma, "Phishing URL detection using LSTM based ensemble learning approaches," Int. J. Comput. Netw. Commun. (IJCNC), vol. 15, no. 1, pp. 17-33, 2023, doi: 10.5121/ijcnc.2023.15102.

[2] R. Zieni, L. Massari, and M. C. Calzarossa, "Phishing or not phishing? A survey on the detection of phishing websites," IEEE Access, vol. 11, pp. 18499-18519, 2023, doi: 10.1109/ACCESS.2023.3247135.

[3] V. Rani, V. Bardeja, and H. Sharma, "Real-time phishing website detection via mutual information-driven feature selection and

random forest ensemble classification," J. Commun., vol. 21, no. 6, pp. 1-8, 2026, doi: 10.43681/JOCS21.06/2605.

[4] N. T. Lam, "Developing a framework for detecting phishing URLs using machine learning," Int. J. Emerg. Technol. Adv. Eng., vol. 11, no. 11, pp. 61-67, 2021, doi: 10.46338/ijetae1121_08.

[5] T. M. Muhammad et al., "Behavioural feature engineering for phishing detection: The case of redirection patterns," J. Behav. Inform., Digit. Humanit. Develop. Res., vol. 11, no. 2, pp. 106-114, 2025, doi: 10.22624/AIMS/BHI/V11N2P210.

[6] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Syst. Appl., vol. 117, pp. 345-357, 2019, doi: 10.1016/j.eswa.2018.09.029.

[7] A. Karim, M. Shahroz, K. Mustofa, S. B. Belhaouari, and S. R. K. Joga, "Phishing detection system through hybrid machine learning based on URL," IEEE Access, vol. 11, pp. 36805-36822, 2023, doi: 10.1109/ACCESS.2023.3252366.

[8] S. Kumar, A. Faizan, A. Viinikainen, and T. Hamalainen, "MLSPD - machine learning based spam and phishing detection," in Computational Data and Social Networks (Lecture Notes Comput. Sci., vol. 11280). Springer, 2018, doi: 10.1007/978-3-030-04648-4_43.

[9] P. Bountakas and C. Xenakis, "HELPHED: Hybrid ensemble learning PHishing email detection," J. Netw. Comput. Appl., vol. 210, Art. no. 103545, 2023, doi: 10.1016/j.jnca.2022.103545.

[10] M. N. Yeasmin, Md. A. R. Refat, B. C. Singh, Z. Alom, Z. Aung, and M. A. Azim, "EnLeM: Ensemble learning-based model to detect phishing websites," EURASIP J. Inf. Secur., vol. 2026, Art. no. 1, 2026, doi: 10.1186/s13635-025-00219-1.

AI Turnitin

CRIS_formatted.pdf

The Northcap University

Document Details

Submission ID
trn:oid:::1:823695006

Submission Date
Oct 5, 2026, 4:50 PM GMT

Download Date
Oct 5, 2026, 4:51 PM GMT

File Name
CRIS_formatted.pdf

File Size
231.9 KB

7 Pages

4,684 Words

27,128 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

CRIS: A Hybrid Machine Learning Framework for Phishing URL and Email Detection

Vinothagan K
UG Scholar
230701901@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering.
Rajalakshmi Engineering College
Thandalam, Chennai, India

Janarthanan B
UG Scholar
230701125@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam, Chennai, India

Mr. Salman Latheef T A
Assistant Professor
salmanlatheef.ta@rajalakshmi.edu.in.
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India

Praveen S
UG Scholar
230701245@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam, Chennai, India

Poovarasan T
UG Scholar
230701228@rajalakshmi.edu.in.
Dept. of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam, Chennai, India

Abstract

Phishing remains one of the most enduring and affordable attack vectors in the field of cybersecurity, targeting users with deceptive URLs and fake electronic messages instead of attacking vulnerable software components. A considerable number of prior studies distinguish between phishing URL detection and phishing message detection as two separate challenges and evaluate the algorithms developed separately in carefully crafted single-modality datasets. The paper discusses the detection module of the Cyber Risk Analysis and Intelligence System (CRIS), which represents a hybrid detection architecture based on combining supervised learning and transparent heuristic rule-based approach to analyzing both Uniform Resource Locator (URL) and electronic messages within the same Flask-based web application. Specifically, the CRIS uses a Random Forest classifier for detecting phishing using thirteen structural URL features and risk scoring hand-coded function, while for message detection – a Logistic Regression algorithm with TF-IDF features. The combination of models' results and heuristic reasoning provides a transparent risk conclusion with a corresponding human-readable justification. The paper outlines the detection methodology, system architecture, dashboard implementation, and reporting process and suggests a framework for future dataset-based evaluation of the system. No artificial performance measures are provided; the contribution of the paper is methodological and architectural. The real-time interception of phishing URLs at the moment of user clicking, based on the above detection core, is discussed in a separate work and is out of scope of the paper..

Index Terms

Phishing, Machine Learning, Random Forest, Logistic Regression, TF-IDF, Cyber Security, URLs, E-Mail Security

I. Introduction

Phishing attacks differ from attacks that exploit software vulnerabilities by fooling the victim into believing they are communicating with a trustworthy entity in order to get credentials, financial information, or other data out of the victim, or to get them to perform some other action on behalf of the attacker. Since the attack exploits the human decision-making process, which is not a vulnerability in the technological infrastructure, the strengthening of the latter does little good to thwart it. In any case, only two methods of delivery prove prevalent in practice - links inserted into webpages, advertisements, or other forms of messaging and messages that prompt their recipient for sensitive data or steer them towards a malicious website - but are researched separately from each other as distinct problems, despite sharing a common root.

A considerable number of contributions have made in the form of machine-learning classifiers for phishing URLs and phishing emails, all of which achieve very high results on publicly available datasets [1]-[10]. Nevertheless, there are three practical drawbacks shared by a majority of these works. Firstly, among the ten papers discussed in this essay, eight focus on detecting phishing websites and URLs, one detects emails for phishing attacks [9] and one detects both spam and phishing [8]; however, none, to the best of our knowledge, combines URL and message detection in one system. Second, the great majority of these studies evaluate a classifier offline on a held-out test partition, as for example in [1], [4], [6], [9], [10], rather than as a component embedded in an operational, user-facing workflow. Third, many systems report a classification label without an accompanying, human-readable justification for that label, which limits the extent to which an end user can reasonably trust or act on the output.

This paper presents CRIS (Cyber Risk Analysis and Intelligence System), a detection platform that combines a Random Forest classifier for structural URL analysis and a TF-IDF/Logistic Regression pipeline for message analysis within a single web application, together with a transparent, hand-coded rule-based scoring layer so that every verdict is accompanied by human-readable justification rather than an opaque probability alone. This detection core is also designed to serve as the foundation for a real-time interception workflow, addressed in a separate study outside the scope of the present paper. The present paper is scoped exclusively to the detection core itself, its methodology, implementation, and evaluation plan.

Section II surveys prior ensemble, deep-learning, and real-time approaches to phishing URL and email detection, and

Section III defines the problem and the project's objectives. Section IV presents the proposed system and its architecture, while Section V details the detection methodology for URLs and messages. Implementation, testing, and evaluation are covered in Sections VI through VIII, and the paper closes with discussion, limitations, future work, contributions, and a conclusion (Sections IX-XIII).

II. Related Work

Most prior work bearing on the CRIS detection core applies ensemble or feature-engineering techniques to URL and website classification. A smaller set of studies targets email or messages, and one survey provides wider context; the literature is therefore organized into the four groups below.

A. Ensemble and Feature-Engineering-Based Phishing URL and Website Detection

Several works in this space build ensembles directly on top of lexical or behavioral URL features. In one line of work, bagging and stacking are applied across multiple Long Short-Term Memory (LSTM) models for phishing URL classification, with mutual-information feature selection narrowing the lexical feature set; the stacking variant came out ahead of the other machine-learning baselines tested in that study [1]. Mutual-information-driven feature selection resurfaces in a Random-Forest-based ensemble aimed specifically at real-time phishing-website detection [3] - a design goal that lines up closely with the real-time, point-of-click motivation behind the broader CRIS project. A large-scale study takes a different angle, pairing Random Forest and Support Vector Machine classifiers with behavior-based URL features and testing the approach against roughly 470,000 URLs sourced from Vietnamese phishing-reporting channels [4]. Among seven classification algorithms compared in a real-time anti-phishing system built on natural-language-processing-derived URL features, a Random Forest classifier using only the NLP-based features came out on top, reaching 97.98% accuracy [6]. A hybrid machine-learning system operating purely on URL-derived features is proposed elsewhere [7]. Last but not least, the EnLeM method uses the voting scheme. That means that there are three classifiers: decision tree, random forest, and k-nearest neighbor that make their own vote after the feature selection procedure, and the overall result is supposed to be better and faster than other methods [10]. A pattern worth noting across this group: Random Forest keeps coming out as a strong, computationally cheap base learner for structural or lexical URL features [3], [4], [6], [10], which is a large part of why it anchors the URL-classification component of CRIS (Section V-B).

B. Feature Engineering and Specialized Detection Approaches

Two further studies address more specialized angles. Muhammad et al. examined redirection-chain patterns as a behavioral feature category for phishing detection [5], a feature category directly analogous to the redirect-pattern feature used within the CRIS URL feature set (Table I, feature 12). MLSPD [8] handles spam and phishing detection jointly in one machine-learning framework; within the literature reviewed here, it is the one work besides [9] whose scope reaches message content such as spam, which makes it relevant to CRIS's own combination of URL and message analysis.

C. Phishing Email Detection

HELPHED takes a hybrid ensemble approach to phishing email detection, running two base learners in parallel over a mix of textual and structural features and combining their outputs through soft voting; it was tested on roughly 32,000 legitimate and 3,500 phishing emails [9]. Of the ten papers reviewed for this study, HELPHED is the only one dedicated to email detection; the rest sit on the URL/website side, apart from the joint spam-and-phishing scope of MLSPD [8]. Section II-D returns to this imbalance, and Section XI flags it as a gap worth closing with further literature review.

D. Surveys and Broader Context

Zieni et al. reviewed the existing literature on approaches to detect phishing websites [2]. This review gives wider perspective of the detection approaches, unlike the approaches reviewed in Sections II-A to II-C, which Do not in themselves suggest a detection approach.

E. Positioning of CRIS Relative to Existing Work

The reviewed literature demonstrates that ensemble and feature-engineering-based machine learning, most commonly built around Random Forest, achieves strong classification accuracy for phishing URL and website detection [1], [3], [4], [6], [7], [10], a finding consistent with CRIS's own selection of Random Forest for URL classification. CRIS differs from this literature primarily in scope and integration rather than in classification technique: rather than optimizing URL-classification accuracy in isolation, CRIS combines URL analysis with message and email analysis within a single operational platform, with each pipeline paired with a transparent rule-based scoring layer so that every verdict is accompanied by interpretable justification rather than a bare label. As noted in Section II-C, only HELPHED [9] among the reviewed papers is dedicated to email detection, and no reviewed work combines URL and message analysis within one integrated platform that also justifies each verdict; this combination, together with the rule-based interpretability layer, constitutes the primary point of differentiation for CRIS. CRIS should therefore be understood as complementary to, rather than a replacement for, the classification techniques reported in [1]-[10]; the ensemble and feature-selection methods surveyed in Section II-A in particular represent a direct avenue for improving the URL-classification component of CRIS, as discussed in Section XI.

III. Problem Statement and Objectives

A. Problem Statement

Phishing attacks are delivered to users via two different but loosely connected channels - malicious URLs and fraud electronic messages. Most of the current academic systems used to detect such attacks are developed as offline classifiers and are tested against a single channel. Moreover, the output of these systems is not easily interpretable by a human being. Thus, the problem that we try to solve in this project can be stated as follows:

Is it possible to merge URL- and message-based phishing detection into one system whose decision will be understandable by humans?

B. Objectives

- Create a machine-learning classifier using structural URL properties to detect phishing links.

- Create another machine-learning classifier based on text to identify phishing messages/emails.
- Couple decisions from both classifiers with a clear scoring system based on explicit rules so that all judgments have an easily understandable explanation.
- Cre ate a central interface for the reporting and history functionality.
- Specify a methodical way of evaluation that does not include fake statistics to facilitate further quantitative assessment of the system.
- Ensure that the detection component is independent enough to allow integration into a real-time interception flow at a later stage of development (the process is considered separately)..

IV. Proposed System

CRIS is a centralised detection system implemented using a Flask-based web application that can be accessed via a manual web dashboard. Every URL or message received from the web dashboard undergoes evaluation using the detection algorithms explained in Section V, and the output generated from the evaluation is then recorded and exported.

A. System Architecture

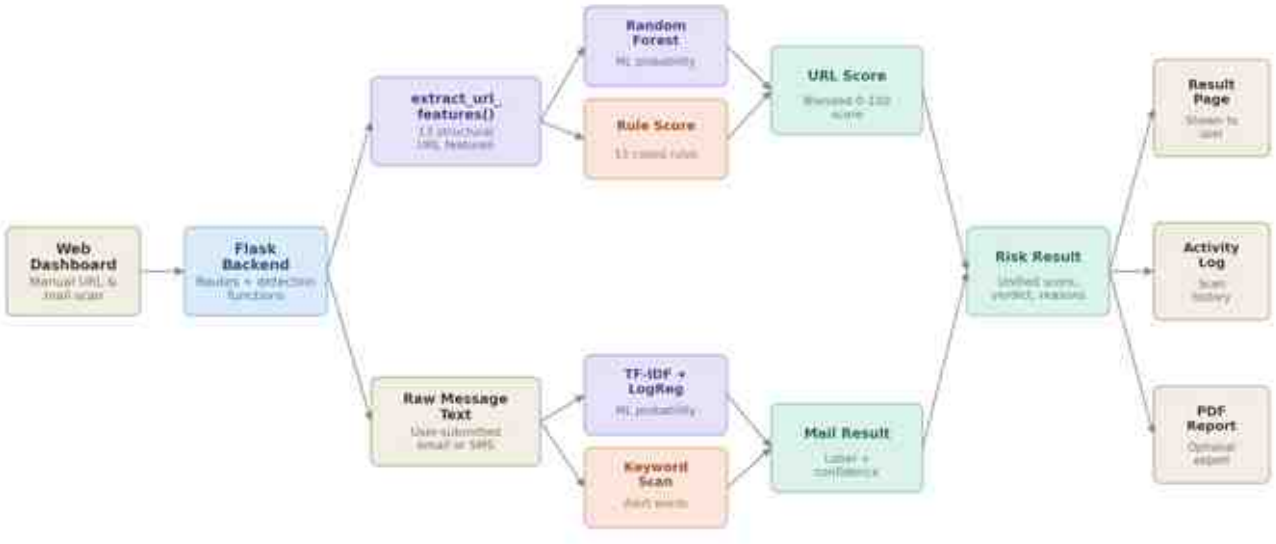


Fig. 1. CRIS detection-core architecture.

B. Manual Analysis Interface

CRIS users will access the tool via a website interface that has two key interfaces; the URL Inspector and the Mail/Message Analyzer. The two tools require a manually inputted URL or message in order to provide its verdict. There are additional screens that make up the interface including the Control Center, which provides quick scan results; Insights, which highlights the frequency of risky keywords; History, which provides previous scan results; and Reports, which provides previous reports.

V. Detection Methodology

URL analysis and message analysis in CRIS are considered independent binary classification tasks that involve both a machine learning model and a rule-based scoring system.

A. URL Feature Engineering

After parsing, all the candidate URLs are represented in a structure feature vector of constant length, shown in Table I below.

TABLE I

URL-Derived Features Supplied to the Classifier

#	Feature	Description
1	URL length	Number of characters in the full URL
2	Dot count	Occurrences of the '.' character
3	Digit count	Occurrences of numeric digits
4	Special chars	Non-alphanumeric symbols present
5	HTTPS	Flag set when the HTTPS scheme is used
6	IP address	Flag set when the host is a raw IP
7	Keyword count	Hits on suspicious terms (e.g., "login")
8	Subdomains	Subdomain levels in the host name
9	'@' symbol	Flag set when '@' appears in the URL
10	Hyphen count	Hyphens appearing in host or path
11	Path length	Characters in the path component
12	Redirect	'//' redirect pattern inside the path

#	Feature	Description
13	Query params	Parameters supplied in the query string

B. URL Classification Algorithm

A feature vector of 13 dimensions is then input to a Random Forest algorithm, which contains 100 decision trees with a maximum depth of 10. Random Forest algorithm was chosen because of the consistency of the performance it has shown as a solid and computationally fast baseline classifier for URL features in previous studies [4], [6], [10].

C. Rule-Based URL Risk Scoring

Independently of the machine-learning classifier, a deterministic rule-based function evaluates the same structural features against a fixed set of heuristics to produce a rule-based risk score in the range 0-100 together with a list of human-readable reasons.

D. Score Fusion and Verdict Classification

The final URL risk score combines the rule-based score with the classifier's predicted phishing probability, expressed as:

$$Score = 0.6 \times R + 0.4 \times (P \times 100) \quad (1)$$

where R is the rule-based risk score and P is the Random Forest's predicted phishing probability. The resulting score is mapped to one of three verdicts - Safe, Suspicious, or Dangerous - using fixed thresholds, and is accompanied by both the triggered rule-based reasons and the per-feature importance derived from the trained Random Forest.

E. Message and Email Feature Representation

The content of messages is encoded through a TF-IDF vectorizer, which uses unigrams and bigrams, removes English stop words, and has a maximum vocabulary size of 3,000, chosen for its lightweight and low-latency nature because of the real-time validation purpose of the whole CRIS project..

F. Message Classification Model

Finally, a Logistic Regression classifier with an inverse regularization value of 1.0 classifies the TF-IDF vector. Like the URL pipeline, the output probability of the classifier is used along with a separate set of heuristic factors such as a list of suspicious keywords, use of too many exclamation marks, and commonly used urgent phrases to give reasons for the classification result..

G. Design Rationale for Model Selection

Selection of Random Forest and Logistic Regression has been made in regard to two project-specific design criteria and not due to any presumption of intrinsic superiority of either method in terms of raw prediction accuracy compared to all other methods discussed in Section II. The first criterion is that both methods are cheap in training and inference in terms of computational costs, which is a requirement because of intended real-time interception workflow, which is a topic of another research, where decision must be made within the budget of one navigation event, while the sequence-based methods, such as the ensemble trained using LSTM in [1], demonstrate the increased training cost and inference overhead introduced by the more complex model. Second, both methods have an easily interpretable internal structure in the form of feature importance in case of Random Forest or coefficient of each term for Logistic Regression with the fixed TF-IDF vocabulary. Such choice allows building rule-based reasoning layer described in Section V-C and V-F without the need for external explainability method. The above selection doesn't prohibit the replacement of the base learner with a stronger model, including the ensemble and feature selection as discussed in Section II-A and II-B [1], [3], [4], [6], [7], [10] on condition that such substitution would preserve the latency and interpretability criteria..

H. End-to-End Processing Pipeline

Regardless of what classifier is used, all submissions go through the same stages of processing: (i) the input data is obtained by the corresponding Flask route; (ii) the input data is converted to its numerical form – a thirteen-dimensional vector for the URL case, or a TF-IDF vector for the message; (iii) the classifier returns the probability of a class; (iv) the independent rule-based module processes the same input and returns the rule score and list of triggering reasons; (v) in the URL case, the probability from the classifier and the rule score are fused using the formula in Section V-D, while in the message case they are just provided side-by-side; (vi) the final decision, confidence scores, and reasons are put into the common result object; and (vii) this result object is simultaneously sent to the requesting web page, added to the activity log, and passed to the PDF report generation module (Section VI-D).

VI. System Implementation

A. Development Approach

First to be developed was the detection engine of CRIS, which defined URL and message classification, scoring based on rules, and the manual dashboard interface, all of which were a stable and independent structure that would later be reused without change by another entry point, the real-time entry point, covered in a different study.

B. Technology Stack

TABLE II
Software Components Used

Component	Technology / Purpose
Server side	Python with the Flask framework
Machine Learning	scikit-learn (Random Forest, Logistic Regression, TF-IDF) and NumPy
Reporting	ReportLab for PDF-based evaluation reports
Frontend	Jinja2 templates with vanilla CSS/JavaScript
Containerization	Docker and Docker Compose
CI / Quality	GitHub Actions running Black, isort, and mypy

C. Application Modules

The features shown in the web dashboard include the following: a Control Center showing the overall level of detection; a URL Inspector and Mail/Message Analyzer to submit manually; an Insights section that provides aggregated information on keywords and risks; a History view that shows past scans; a Reports section to get PDF evaluation reports; and Settings and About pages..

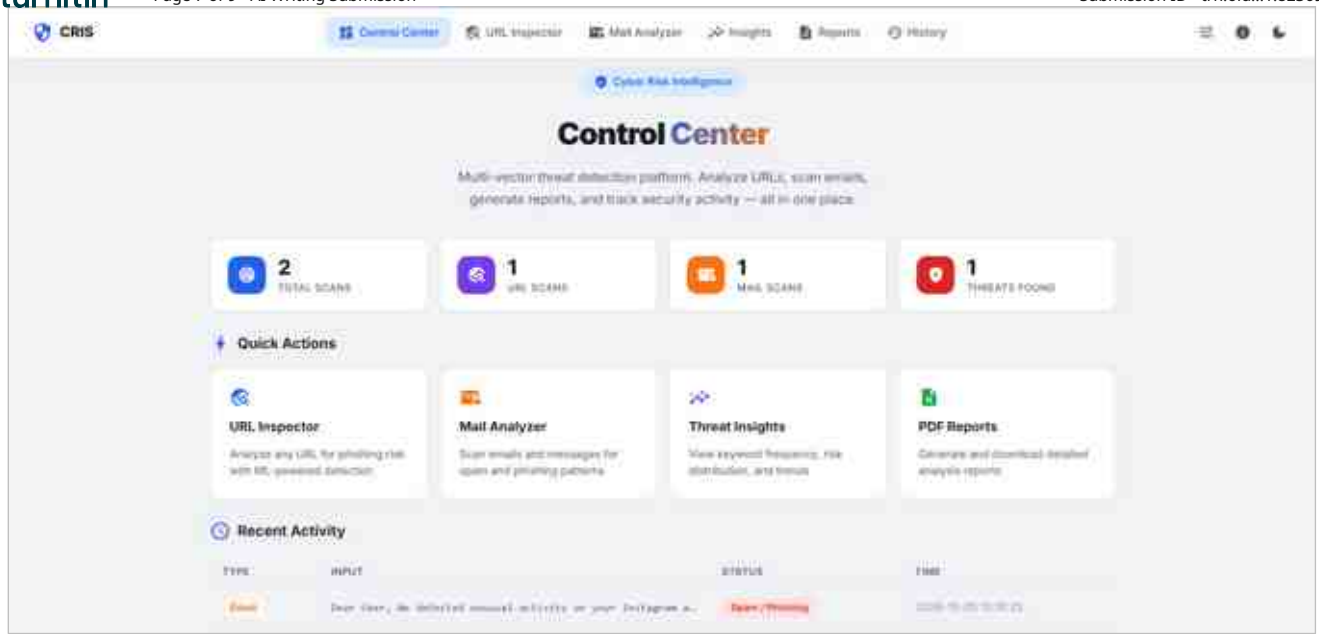


Fig. 2. CRIS Control Center dashboard, showing scan statistics, quick actions, and recent activity.

D. Reporting and Activity Logging

Every evaluation is logged to an activity log that will fill the Control Center view, the History view, and the Insights view and may be optionally exported as a PDF file. The current implementation keeps activities in memory during the lifecycle of the application process but does not store them in a permanent database; this problem is addressed in Section X.

VII. Testing Strategy

Up to now, testing has been done functionally but not formally. All classification and scoring pathways have been put through the system with test URLs and messages, and each run resulted in a full risk analysis - including verdict, confidence scores, and reasons - without any errors.. Unit tests for the individual classifier and feature-extraction functions are not yet in place; they are planned alongside the dataset-driven evaluation described in Section VIII.

VIII. Proposed Evaluation Methodology

No dataset-driven evaluation of the CRIS classifiers has been carried out, which is why this paper quotes no accuracy, precision, recall, or F1 figures. This section instead outlines how such an evaluation should be run.

A. Proposed Dataset Composition

Future evaluation of the URL classifier is intended to use a labeled corpus of real phishing and legitimate URLs, comparable in scale and sourcing to the large, real-world URL collections used in [4] and [6], in place of the synthetic feature distributions currently used for demonstration purposes. Future evaluation of the message classifier is intended to use a real, labeled phishing and legitimate email or message corpus comparable in scale to that used in [9], in place of the small hand-authored example set currently used for demonstration purposes.

B. Proposed Evaluation Protocol and Metrics

The proposed protocol follows conventional supervised-learning evaluation practice: a stratified train/test split (or cross-validation), computation of standard classification metrics on a held-out test partition, and reporting of a confusion matrix for each classifier independently.

TABLE III

Proposed Evaluation Metrics

Metric	Purpose
Accuracy	Fraction of all samples labeled correctly
Precision	Fraction of phishing alerts that were genuine
Recall	Fraction of real phishing cases that were caught
F1-score	Single score combining precision and recall
False-positive rate	Fraction of legitimate items wrongly raised as phishing
False-negative rate	Fraction of phishing cases that slipped through

IX. Discussion

Section II shows that ensembles and engineered-feature classifiers can reach strong accuracy on phishing URL and website data; Sahingoz et al., for instance, report 97.98% accuracy for a Random Forest that uses only natural-language-processing-derived URL features [6]. CRIS does not attempt to exceed such reported figures, and, as stated in Section VIII, does not currently report comparable figures of its own pending a dataset-driven evaluation. The contribution of this work is architectural rather than accuracy-driven: combining URL and message detection within a single platform, with a rule-based interpretability layer that accompanies every machine-learning verdict with human-readable justification - a form of interpretability pursued through a different mechanism than the works reviewed in Section II, the great majority of which report classification performance without an accompanying rule-based justification layer.

based scoring layer in relation to the machine learning classifier. In the URL pipeline, the two signals are numerically combined, meaning that if there were a discrepancy between the heuristic rule scores and those

provided by the classifier, then one gets a numerical score anyway instead of getting a discrepancy signal. In the message pipeline, however, the two signals are shown separately, thus providing insight into any discrepancies between the two scores while sacrificing a unified numerical score. This asymmetric approach to combining the two fusion strategies was deliberately preserved for the purpose of the current paper because unification is an empirical question and depends on the behavior of both on real data, as described in Section VIII.

Another aspect that needs discussion is the dashboard-only interaction model used in this project. While it can be seen as a limitation of the detection core, considering the limited usability of a detection mechanism that can be accessed through submitting a URL or a message, this is a conscious decision for the current stage. The fact is that a separate study should be conducted, in which an automatic entry surface would be developed. The reason is that, due to the separation from the entry surface, the functions of the detection core can be reused without any modifications..

X. Limitations

The current implementation carries several limitations that define the scope of the present contribution and distinguish an architectural demonstration from a production-ready, field-validated system.

First, the URL classifier is currently trained on synthetically generated feature distributions rather than on labeled real-world URLs, and the message classifier is trained on a small, hand-authored set of example messages rather than on a real phishing-message corpus. Consequently, no claim is made regarding real-world generalization until the evaluation described in Section VIII is completed on the datasets proposed there.

Second, both classifiers are retrained in memory each time the application starts, rather than being trained offline once and loaded from a persisted model artifact at startup. This does not affect the correctness of a given run, but it is computationally wasteful at scale and means the exact trained model is not guaranteed to be identical between successive runs, which complicates reproducibility of any future reported result.

Third, activity history and generated reports are retained only in memory for the duration of the running application process, rather than in a persistent database; this history is lost whenever the process is restarted, which is acceptable for the current demonstration scope but would not be acceptable for a deployed system expected to retain an audit trail across sessions.

Finally, no quantitative classification results are yet available, as stated in Section VIII, and this paper is scoped exclusively to the detection core itself; a real-time, interactive extension of this system is left entirely to future, separately reported work.

XI. Future Work

Several directions follow from the limitations identified in Section X. The most immediate is replacing the synthetic and hand-authored training data with real-world labeled URL and message datasets and executing the evaluation methodology proposed in Section VIII, since this is the step that would convert the architectural claims made in this paper into measured results.

A second direction is persistence: training the classifiers offline and loading them as saved model artifacts at application startup, together with introducing a persistent database for activity history and reporting data, in place of the current in-memory implementations.

A third direction is evaluating whether the ensemble and feature-selection techniques surveyed in Section II improve classification performance when substituted into the CRIS pipelines, while preserving the latency and interpretability properties discussed in Section V-G.

Finally, the detection core presented in this paper is intended to serve as the foundation for a separately reported extension of the system toward real-time, point-of-click protection; the design and evaluation of that extension are outside the scope of the present paper.

XII. Contributions

Taken together, this work contributes the following.

- An integrated architecture combining independently trained URL and message phishing classifiers with a shared, transparent rule-based scoring layer.
- A score-fusion formulation combining machine-learning probability with rule-based scoring to produce an interpretable, human-readable verdict.
- A centralized dashboard, activity-logging, and PDF-reporting workflow for reviewing detection activity.
- A structured, non-fabricated evaluation methodology intended to guide future quantitative assessment.
- A self-contained detection core designed to serve as the foundation for a later, separately reported extension of the system.

XIII. Conclusion

This paper has described CRIS, which pairs Random-Forest-based URL classification with TF-IDF/Logistic-Regression-based message classification, and ties both to a transparent rule-based scoring layer inside a single Flask-based platform. In keeping with the academic integrity requirements for this work, no performance results have been fabricated; a structured evaluation methodology is proposed in their place. The detection core described here is meant to underpin a real-time interception extension, which is the subject of a separate, future study.

Acknowledgment

The authors thank Rajalakshmi Engineering College, and in particular its Department of Computer Science and Engineering, for the infrastructure and support provided during this work, and their project guide and faculty mentors for ongoing feedback.

References

[1] B. Banik and A. Sarma, "Phishing URL detection using LSTM based ensemble learning approaches," Int. J. Comput. Netw. Commun. (IJCNC), vol. 15, no. 1, pp. 17-33, 2023, doi: 10.5121/ijcnc.2023.15102.

[2] R. Zieni, L. Massari, and M. C. Calzarossa, "Phishing or not phishing? A survey on the detection of phishing websites," IEEE Access, vol. 11, pp. 18499-18519, 2023, doi: 10.1109/ACCESS.2023.3247135.

[3] V. Rani, V. Bardeja, and H. Sharma, "Real-time phishing website detection via mutual information-driven feature selection and

random forest ensemble classification," J. Commun., vol. 21, no. 6, pp. 1-8, 2026, doi: 10.43681/JOCS21.06/2605.

[4] N. T. Lam, "Developing a framework for detecting phishing URLs using machine learning," Int. J. Emerg. Technol. Adv. Eng., vol. 11, no. 11, pp. 61-67, 2021, doi: 10.46338/ijetae1121_08.

[5] T. M. Muhammad et al., "Behavioural feature engineering for phishing detection: The case of redirection patterns," J. Behav. Inform., Digit. Humanit. Develop. Res., vol. 11, no. 2, pp. 106-114, 2025, doi: 10.22624/AIMS/BHI/V11N2P210.

[6] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Syst. Appl., vol. 117, pp. 345-357, 2019, doi: 10.1016/j.eswa.2018.09.029.

[7] A. Karim, M. Shahroz, K. Mustofa, S. B. Belhaouari, and S. R. K. Joga, "Phishing detection system through hybrid machine learning based on URL," IEEE Access, vol. 11, pp. 36805-36822, 2023, doi: 10.1109/ACCESS.2023.3252366.

[8] S. Kumar, A. Faizan, A. Viinikainen, and T. Hamalainen, "MLSPD - machine learning based spam and phishing detection," in Computational Data and Social Networks (Lecture Notes Comput. Sci., vol. 11280). Springer, 2018, doi: 10.1007/978-3-030-04648-4_43.

[9] P. Bountakas and C. Xenakis, "HELPHED: Hybrid ensemble learning PHishing email detection," J. Netw. Comput. Appl., vol. 210, Art. no. 103545, 2023, doi: 10.1016/j.jnca.2022.103545.

[10] M. N. Yeasmin, Md. A. R. Refat, B. C. Singh, Z. Alom, Z. Aung, and M. A. Azim, "EnLeM: Ensemble learning-based model to detect phishing websites," EURASIP J. Inf. Secur., vol. 2026, Art. no. 1, 2026, doi: 10.1186/s13635-025-00219-1.